

Software-as-a-Graph: Explicit Dependency Graphs Predict Cascading-Failure Impact in Publish-Subscribe Systems Before Deployment

Ibrahim Onuralp Yigit^{a,*}, Feza Buzluca^a

^a*Department of Computer Engineering, Istanbul Technical University, Istanbul, 34469, Turkey*

Abstract

Publish-subscribe middleware decouples components in space and time, and thereby hides the paths along which failures cascade. Architects need to know which components are systemically critical before deployment, when no runtime telemetry exists. We present Software-as-a-Graph (SaG), which turns architecture descriptions into typed multigraphs, derives the hidden dependencies between publishers, subscribers, brokers, hosts and shared libraries, ranks components by cascading-failure impact, and profiles each flagged component along ISO/IEC 25010 reliability and maintainability. We evaluate training-free, graph-learning and hybrid rankers against simulated cascades, under leave-one-scenario-out cross-validation over twelve synthetic architectures and zero-shot on hand-authored models of five open-source systems, with every headline contrast registered before its run. Making dependencies explicit is the decisive step. On SaG's dependency projection, counting a component's dependents ranks its impact at Spearman $\rho = 0.764$ on held-out architectures, above the registered betweenness engine on all twelve and above every learned engine (0.622–0.683). On the system models it reaches 0.86–0.94, without training and in under a tenth of a second. The same projection identifies components whose failure reaches no one with 94% balanced accuracy. Among learned engines, hybrids that correct the closed-form score are the only ones to beat it significantly (11 of 12 folds). Controls trace learned accuracy to per-component features rather than typing, message passing or QoS edge encodings, and models trained only on synthetic data transfer to the system models ($\rho = 0.757$ –0.831). Registered controls show that ranking needs no declared QoS contracts. Cascading-failure risk thus becomes measurable from architecture models alone.

Keywords: Cascading failures, publish-subscribe, dependency graphs, software architecture, dependability, graph neural networks, QoS

1. Introduction

1.1. Motivation and Problem

Large distributed systems increasingly communicate through asynchronous publish-subscribe (pub-sub) middleware: ROS 2 in autonomous driving [1], Apache Kafka in enterprise event streams [2], DDS in cyber-physical systems [3], MQTT in IoT [4], and cloud-native microservices [5, 6]. Pub-sub decouples producers and consumers in space, time and synchronization [7]. Components interact through topics and brokers rather than direct references, and deployment-time Quality-of-Service (QoS) policies govern reliability, durability, priority and deadlines.

The same decoupling hides how failures spread. Publishers and subscribers share no direct link, so outages, head-of-line blocking and backpressure propagate along concealed paths through brokers, shared topics, colocated hosts and shared libraries [8, 9]. These failures take two forms. In *sequential cascades*, a failed or slow publisher starves the subscribers that depend on its topics [10]. In *simultaneous blasts*, a

*Corresponding author

Email addresses: yigiti@itu.edu.tr (Ibrahim Onuralp Yigit), buzluca@itu.edu.tr (Feza Buzluca)

shared-library crash or host outage takes down every colocated service at once. Neither architecture diagrams nor static call graphs show these mechanisms, because the dependency between two services exists only through the topic, broker or library they share.

The cheapest time to reduce this risk is before deployment, at design and continuous-integration time [11, 12], when no runtime telemetry exists. The information needed is increasingly available then: Architecture-as-Code manifests, ROS 2 launch files and broker configurations already declare which component publishes and subscribes to which topic, which broker routes it, which host runs it and which libraries it links. What existing practice lacks is a way to turn that declaration into a ranking of systemic risk. Architecture evaluations such as ATAM rely on manual elicitation [13]. Static code analysis inspects services in isolation [14, 15], so a system can have clean code in every service and still be fragile through a hidden single point of failure, a gap between architecture and code that architectural-smell research documents [16, 17]. Chaos engineering needs a provisioned cluster [18]. Centrality on untyped graphs flattens the distinction between topics, libraries and hosts [19, 20]. Learned models could combine structural cues, but it is open whether they add anything once the dependencies are explicit.

1.2. The Software-as-a-Graph (SaG) Approach

Software-as-a-Graph (SaG) is a pre-deployment static analysis framework for event-driven architectures (Figure 1). It (1) models an architecture as a typed, directed multigraph over Applications, Brokers, Topics, Execution Hosts and Shared Libraries (Section 3.1); (2) derives a `DEPENDS_ON` projection that makes the hidden dependencies explicit and distinguishes sequential cascades from simultaneous blasts (Section 3.2); (3) ranks components by cascading-failure impact with training-free, learned and hybrid engines (Sections 4 and 6.2); and (4) profiles flagged components along ISO/IEC 25010 reliability and maintainability sub-characteristics (Section 5). Predictors read only the analysis graph; ground-truth impact comes from simulators that run on the raw structural topology (Section 4.4).

The central thesis is that making dependencies explicit is the decisive investment. Once the projection exists, the question “whose failure reaches furthest?” becomes a question about how many components depend on whom, which a count answers directly and which learned engines, hybrids and QoS weights can then be judged against.

1.3. Research Questions

- **RQ1 (Ranking accuracy):** *How accurately do training-free, learned and hybrid engines on SaG’s graphs rank components by cascading-failure impact on unseen architectures, and what drives the accuracy of the training-free engines?*
- **RQ2 (What learning needs):** *Where does the learned engines’ accuracy come from—relation-specific (typed) parameters, message passing, the QoS edge encoding, or the per-component features SaG extracts—once model capacity and edge-channel width are matched?*
- **RQ3 (Transfer):** *How well do these rankers transfer to independently authored models of five open-source systems?*
- **RQ4 (Cost):** *What does the analysis cost at CI/CD time, and how does it compare with running the simulation directly?*

Every headline contrast was registered with a decision rule before its results existed: the primary contrast, the matched control, both hybrid engines, and the training-free dependency counts with their QoS-attribution controls. Section 6.4 and Supplementary Section S24 log every amendment. The attribution controls of Section 7.2, which separate message passing and the QoS node features from the edge encoding, were added after the matched control’s result; they are exploratory and are logged as Amendment 8.

1.4. Key Findings at a Glance

1. **Counting dependents ranks failure impact best.** On SaG’s dependency projection, the number of a component’s dependents ranks its simulated cascade impact at Spearman $\rho = 0.764$ on held-out architectures, above the registered betweenness engine on all twelve (+0.211) and above every learned and hybrid engine (0.622–0.683). It identifies the components whose failure reaches no one with 94% balanced accuracy (Section 7.1).

2. **The dependency projection, not QoS weighting, carries the signal.** Registered controls attribute the closed-form engine’s gain over the registered centrality baseline ($0.349 \rightarrow 0.553$) to the projection: constant or permuted topic weights rank as well as the declared QoS contracts (Section 7.1.2).
3. **Learning helps over betweenness, and its accuracy comes from SaG’s features.** Hybrid engines are the only learned engines that significantly beat the closed-form engine ($+0.103$ and $+0.130$, 11 of 12 folds). Matched controls show that relation-specific parameters, message passing and the QoS edge encoding add nothing measurable, and a gradient-boosted regressor on the same per-component features matches the neural engines (0.642; Sections 7.1 and 7.2).
4. **Rankings transfer.** Trained only on synthetic data, learned models transfer to five open-source system models at $\rho = 0.757\text{--}0.831$, and dependency counts reach 0.86–0.94 with no training at all (Section 7.3).
5. **The best ranker is the cheapest.** Deriving the projection and counting dependents takes under a tenth of a second on the largest architecture, against seconds for the simulator and minutes for the learned engines’ features (Section 7.4).

1.5. Contributions

1. **A typed architecture model with an explicit dependency projection** for pub-sub systems, whose six derivation rules turn publish, subscribe, routing, hosting and library relations into dependencies and separate sequential cascades from simultaneous blasts (Section 3). On it, training-free dependency counts are accurate, transferable and effectively free.
2. **A controlled comparison of what learning adds** (Sections 4 and 7): heterogeneous and homogeneous graph neural networks at matched capacity, directionality and capacity controls, a feature-only regressor, hybrid engines that correct a closed-form prior, and training-free dependency counts, all under the same leave-one-scenario-out protocol and zero-shot transfer.
3. **Registered attribution of where accuracy comes from**, including controls that separate the dependency projection from the QoS contracts and that overturned an earlier interpretation of this study (Sections 7.1.2 and 6.4).
4. **A standards-grounded explanation layer** (Section 5) that profiles each flagged component along ISO/IEC 25010 Availability, Fault Tolerance and Maintainability and names a remediation class, presented as a design proposal for validation.
5. **An open benchmark and replication package** in the spirit of the JSS Open Science initiative: seventeen architectures totaling 2,812 components, twelve of which regenerate byte-identically from committed configurations, a torch-free harness for every training-free result, and a reconciler that mechanically checks every reported table value against the released artifacts.

A previous conference paper [21] introduced the preliminary multigraph and deterministic quality model on synthetic topologies. This paper adds the learned, hybrid and dependency-count engines, the registered leave-one-scenario-out and zero-shot evaluation, the matched and QoS-attribution controls, and the cost profile, and it repositions that quality model as the explanation layer.

Section 2 reviews related work, Sections 3–5 present the model, engines and explanation layer, Sections 6–7 the evaluation, Section 8 the implications and threats, and Section 9 concludes.

2. Related Work

2.1. Dependability Analysis of Distributed Systems

Runtime approaches to dependability, such as broker clustering, backpressure, autoscaling, failover and chaos engineering [18], require a running cluster, can disrupt service, and consume substantial compute. That compute is itself a concern of green software engineering [22, 23, 24]. Architecture-based reliability prediction has a long history. Cheung’s absorbing Markov chain [25] and the state-, path- and additive models surveyed by Goseva-Popstojanova and Trivedi [26] and Immonen and Niemelä [27] are examples, as are the Palladio Component Model [28] and its reliability extension, which propagates failures through component

usage profiles [29], layered queueing networks [30], the AADL Error Model Annex [31] and model-based failure-propagation analyses such as HiP-HOPS [32]. These approaches answer broader questions but need operational profiles and failure rates that are unavailable at commit time. SaG asks a narrower question from manifests alone: whose failure reaches furthest in the declared topology? Test-time fault injection, such as Filibuster’s service-level injection [33] and lineage-driven fault injection [34], avoids production traffic but still executes the services and their tests; SaG needs neither.

Telemetry-driven methods diagnose faults in running microservices. Examples are Seer [35] and Sage [36], MicroRCA [37], TraceRCA [38], the causal-inference method MicroCause [39], and the GNN-based DeepTraLog [40] and Eadro [41] (reviewed in [42, 43]). Trace studies at production scale show how dense and dynamic service dependency graphs are [44]. In synchronous microservices, cascades stem from thread-pool exhaustion, RPC timeouts and retry storms [45]. In pub-sub systems they spread through queue saturation and message starvation. All of these methods need traces or metrics from a live system; SaG works before any code runs. SaG’s question is also that of change impact analysis [46] and of dependency management in microservice fleets [47]. Ranking by impact relates to learning-to-rank defect prediction [48], which optimizes the ranking measure directly, as our listwise loss does (Section 4.2).

2.2. Static Code Analysis and Static System Analysis

Static code analysis (SCA) tools such as SonarQube [14] measure complexity [49], cohesion and coupling [15, 50] within individual services to flag defect-prone modules [51, 52, 53, 54]. SCA cannot see inter-service messaging, broker saturation or cross-host propagation. Architecture recovery tools reconstruct system-level structure from code [55, 56], HAROS [57] extracts the ROS computation graph statically, and ROSDiscover [58] detects run-time architecture misconfigurations in ROS systems before launch. SaG’s static system analysis (SSA) uses the declared topology instead, propagating code-level metrics across architectural dependencies. This lets teams find structural anti-patterns [59, 60] and architectural technical debt [61] in CI/CD [62, 63].

2.3. Quality Models and Multi-Criteria Evaluation

ISO/IEC 25010:2023 [64] and ISO/IEC 25019:2023 [65] define product quality and quality in use. SaG covers the characteristics derivable from deployment topology, namely Availability, Fault Tolerance and the Maintainability sub-characteristics (Section 5.1), and links internal structural quality to external dependability [66, 67]. Aggregating metrics into an auditable score is a multi-criteria decision problem, for which the Analytic Hierarchy Process (AHP) [68] is standard. AHP’s consistency ratio detects inconsistent judgments but not matrices back-filled from a chosen answer, a distinction this study reports for its weights (Supplementary Section S4).

2.4. Graph Learning and Explainability

Centrality indices [19, 20, 69, 70] and cascade models of network robustness [10, 8, 9] assume homogeneous, usually undirected graphs. A single untyped score conflates structurally different elements, such as topics, libraries and hosts, and cannot say *why* a component is critical. Learned node-importance methods, including FINDER [71], DrBC [72] and the cascading-failure benchmark PowerGraph [73], share the homogeneity assumption and are usually judged against centrality indices; we additionally judge learned engines against simple dependency counts on the same graph (Section 7.1). Homogeneous GNNs (GCN [74], GraphSAGE [75], GAT [76]) discard relation identity unless it is supplied as a feature. Heterogeneous GNNs (RGCN [77], HAN [78], HGT [79], MAGNN [80]) learn relation-specific transformations. We evaluate both families under matched capacity (Section 7.2). Khodabandeh et al. [81] apply graph attention to microservice call graphs to predict future interactions. We instead predict the impact of removing a node from a declared topology. GNN explainers such as GNNExplainer [82] and PGExplainer [83] explain models in terms of their internal features. SaG’s explanation layer (Section 5) instead names ISO/IEC quality sub-characteristics and a remediation class for each flagged component.

3. The Software-as-a-Graph (SaG) Architectural Model

Figure 1 presents the end-to-end architecture of the SaG framework. The shared front end processes Architecture-as-Code manifests, constructs a typed multigraph, projects implicit runtime interactions onto a logical dependency layer, and extracts typed node properties. These features feed the ranking engines (Section 4) and, separately and with no shared parameters, the explanation layer (Section 5).

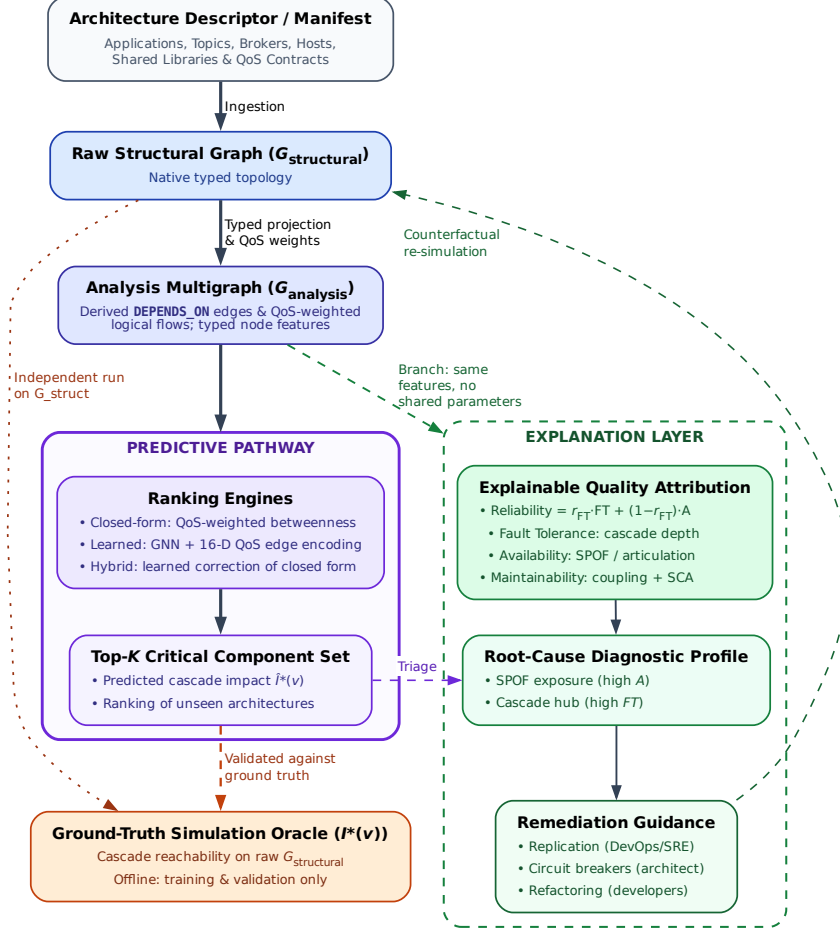


Figure 1: End-to-end architecture of the SaG framework. The predictive pathway runs down the centre: manifest ingestion, typed multigraph, QoS-weighted **DEPENDS_ON** projection with typed node properties, the ranking engines (closed-form, learned and hybrid; Figure 3), and the ranked critical set. The dashed edge marks the ground-truth simulation oracle, which operates only on $G_{\text{structural}}$, trains the predictor offline and takes no part in inference. The explanation layer re-enters from the analysis multigraph and shares no parameters with the predictor; it profiles only the components the ranking flags (triage), rather than receiving the predictor’s outputs.

3.1. Formal Multigraph Definition

A distributed system is described as a typed, weighted, directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \quad (1)$$

where:

- $V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{host}} \cup V_{\text{lib}}$ holds the five entity types $\mathcal{T}_V$ of Table 1; V_{host} denotes physical or virtual *Execution Hosts*.

- E is the set of directed edges, and $\tau_V : V \rightarrow \mathcal{T}_V$ and $\tau_E : E \rightarrow \mathcal{T}_E$ assign entity and relation types.
- $w_V : V \rightarrow (0, 1]$ and $w_E : E \rightarrow (0, 1]$ weight entity criticality and connection strength. For Applications and Libraries, $w_V(v) = 1 - \text{CQP}(v)$, where CQP is a code-quality penalty computed from static code metrics (lines of code, cyclomatic complexity, coupling and cohesion); otherwise $w_V(v) = 1.0$.

Table 1: Entity types and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	Architectural Role	Concrete System Examples
Application (V_{app})	Process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	/sensor/lidar, orders.payment.completed
Execution Host (V_{host})	Physical or virtualized execution environment	Bare-metal server, Kubernetes worker, Cloud VM
Library (V_{lib})	Shared software package or runtime dependency	Kafka client, OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	Direction	Semantic Meaning
PUBLISHES_TO / SUBSCRIBES_TO	App/Library $\rightarrow$ Topic	Component publishes to / consumes from topic
ROUTES	Broker $\rightarrow$ Topic	Broker manages and routes topic traffic
RUNS_ON	App/Broker $\rightarrow$ Host	Process is hosted on host
CONNECTS_TO	Host $\rightarrow$ Host	Network link between hosts
USES	App $\rightarrow$ Library	Application links to shared library

Table 2: Notation used throughout. Entity and edge types: Table 1; simulation oracles: Table 4.

Symbol	Meaning
$G_{\text{structural}}$	Raw multigraph; read only by the simulation oracles
G_{analysis}	Multigraph plus the derived DEPENDS_ON projection; predictor input
V_{app}	Application nodes; the scored population
$w(t), w(e)$	QoS topic weight; edge weight
$I^*(v)$	Primary cascade oracle (Section 4.3)
$I_{\text{comp}}, I_{\text{dyn}}$	Further oracles (Table 4)
$Q(v)$	RM composite quality score of the explanation layer
$\rho, \rho_{>0}$	Spearman ρ on the full population; on the active stratum ($I^* > 0$)
Overlap@ K	Top- K set overlap, $K = \text{round}(0.20 V_{\text{app}})$

3.2. QoS-Aware Weights and Logical Dependency Derivation

A link’s strength depends on its QoS contract: a **RELIABLE** topic with **TRANSIENT_LOCAL** durability couples services more strongly than a **BEST_EFFORT** telemetry stream. Each topic t therefore carries a weight $w(t) \in (0, 1]$ combining its declared QoS with payload size and publication frequency:

$$w(t) = \alpha_{\text{top}} \cdot \text{QoS}(t) + \beta_{\text{top}} \cdot \text{SizeNorm}(t) + \gamma_{\text{top}} \cdot \text{FreqNorm}(t), \quad (\alpha_{\text{top}}, \beta_{\text{top}}, \gamma_{\text{top}}) = (0.75, 0.15, 0.10) \quad (2)$$

where the QoS term is an AHP-weighted aggregate of the declared contract:

$$\text{QoS}(t) = w_{\text{rel}} \cdot q_{\text{rel}} + w_{\text{dur}} \cdot q_{\text{dur}} + w_{\text{prio}} \cdot q_{\text{prio}}, \quad (w_{\text{rel}}, w_{\text{dur}}, w_{\text{prio}}) = (0.24, 0.62, 0.14) \quad (3)$$

Here $q_{\text{rel}} \in \{0, 1\}$ (best-effort, reliable), $q_{\text{dur}} \in \{0, 0.5, 1\}$ (volatile, transient-local, persistent) and $q_{\text{prio}} \in \{0, 0.5, 1\}$ (low, medium, high). Durability carries the highest weight because it determines whether state and message history survive restarts. The sub-weights come from a Saaty pairwise matrix with genuine second-eigenvalue spread and $CR = 0.016$ (Supplementary Section S4). Size and frequency are log-compressed and clamped:

$$\text{SizeNorm}(t) = \min \left(1.0, \frac{\log_2(1 + B(t))}{20} \right), \quad \text{FreqNorm}(t) = \min \left(1.0, \frac{\log_{10}(1 + F(t))}{3} \right) \quad (4)$$

where $B(t)$ is the payload in bytes and $F(t)$ the publication frequency in Hz; the normalizers make the terms saturate at a 1 MiB payload and a 1 kHz stream, the upper ends of the corpus envelopes. $w(t)$

is clamped to $[0.01, 1]$ so that best-effort edges remain visible, and every PUBLISHES_TO, SUBSCRIBES_TO and ROUTES edge of t carries $w_E(e) = w(t)$ and the topic's QoS vector. The $(\alpha_{\text{top}}, \beta_{\text{top}}, \gamma_{\text{top}})$ split is a documented choice rather than a sensitive parameter: no point of its simplex changes any reported ordering (Supplementary Section S1). On the reachability oracle used in this study, the QoS *content* of $w(t)$ does not improve ranking either: fixing every topic weight to a constant, or permuting QoS profiles across topics, ranks as well as the declared contracts do (Section 7.1.2). The weights are kept because they are what the explanation layer and the learned engines' edge channel read.

3.2.1. Logical Dependency Projection (DEPENDS_ON)

Structural edges do not capture implicit runtime dependencies: a subscriber depends on a publisher, yet no edge joins them. SaG therefore derives one semantic relation, DEPENDS_ON, directed from *dependent* to *dependency* ("if the target fails, the source is impacted"), by the six rules of Table 3. Its weight $w \in (0, 1]$ expresses how likely a disruption of the dependency is to reach the dependent.

Table 3: The six DEPENDS_ON logical dependency projection rules.

Rule	Dependency Category	Structural Pattern (Dependent $\rightarrow$ Dependency)	Derived Weight (w)
1	app_to_app	Subscriber $\rightarrow$ Publisher (via shared Topic, incl. transitive USES)	$1 - \prod_{t \in T} (1 - w(t))$
2	app_to_broker	Publisher/Subscriber $\rightarrow$ Broker routing its topics	$1 - \prod_{t \in T} (1 - w(t))$
3	host_to_host	Host $\rightarrow$ Host (lifted from inter-host app dependencies)	$\max_{u \in \text{hosted}(h_1), v \in \text{hosted}(h_2)} w_{\text{DEPENDS_ON}}(u \rightarrow v)$
4	host_to_broker	Host $\rightarrow$ Broker (lifted from hosted app dependencies)	$\max_{u \in \text{hosted}(h)} w_{\text{DEPENDS_ON}}(u \rightarrow b)$
5	app_to_lib	Application $\rightarrow$ Shared Library it USES	$H(w_V(\text{app}), w_V(\text{lib}))$
6	broker_to_broker	Broker $\leftrightarrow$ Broker (shared physical fault-domain colocation, symmetric)	$w_V(\text{host})$

Rules 1 and 2 combine the topics T joining a pair by probabilistic union rather than maximum [84, 85, 86], so parallel failure paths always increase coupling. Rule 5 uses the harmonic mean $H(x, y) = 2xy/(x + y)$ [87], and Rules 3 and 4 lift dependencies to hosts by maximum.

Sequential cascades and simultaneous blasts. Rule 1 captures sequential cascades, in which a failed publisher starves subscribers through queues and topic buffers. Rule 5 captures simultaneous blasts, in which a crashed library or host takes down every consumer at once. Untyped graphs collapse the two into indistinguishable edges. Rule 6, the only symmetric rule, joins brokers colocated on a host, which share its failure domain. Figure 2 shows both mechanisms on a seven-entity example.

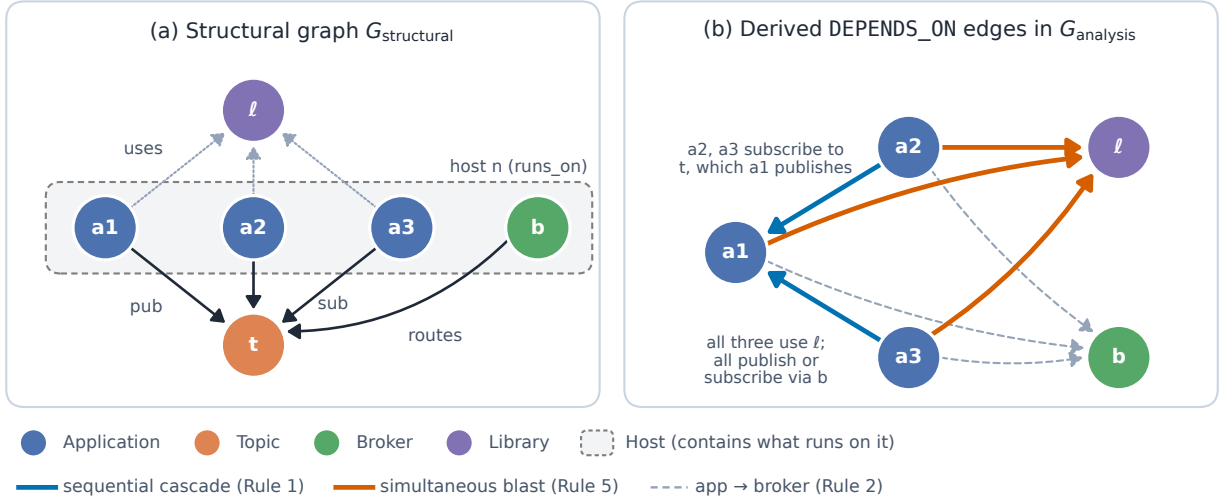


Figure 2: Running example. (a) Three applications share topic t (routed by broker b) and library l , and all run on host n . No structural edge joins two applications. (b) The derived DEPENDS_ON edges make the hidden dependencies explicit: the subscribers $a2, a3$ depend on the publisher $a1$ (Rule 1, a sequential cascade through the topic), every application depends on l (Rule 5, a simultaneous blast if l fails), and each application depends on the broker routing its topic (Rule 2). Simulation oracles run on view (a) only; predictors read view (b).

3.3. Dual Graph Views

The **structural graph** $G_{\text{structural}}$ is the raw deployment topology. The **analysis graph** G_{analysis} adds the derived, QoS-weighted **DEPENDS_ON** edges and the code metrics (Figure 2). All predictor features are computed on G_{analysis} , while simulation oracles run only on $G_{\text{structural}}$ (Section 4.4); Rule 6 therefore cannot influence any label. The primary oracle re-derives Rules 1 and 5 from the raw edges for its own propagation step, so the two views share the dependency semantics even though no derived edge crosses between them (Section 4.4).

3.4. Typed Node Feature Encoding

Both the predictive pathway (Section 4) and the explanation layer (Section 5) read the same typed node properties from G_{analysis} : the predictor projects them per entity type before message passing, the explanation layer aggregates them into a quality profile. All five entity types share a deterministic 18-dimensional base block of topological metrics, each normalized to $[0, 1]$ within its graph so that raw graph size does not drive cross-scenario transfer. The block comprises PageRank and Reverse PageRank; betweenness, closeness and eigenvector centrality; in- and out-degree; clustering; undirected and directed articulation scores; the bridge ratio; the node QoS weight and QoS-weighted in- and out-degree; multi-path coupling; path complexity; fan-out criticality; and the Connectivity Degradation Index (CDI), which removes each node and measures the resulting connectivity loss (full schema: Supplementary Section S11). Normalized in-degree on the dependency graph (index 5), which as a stand-alone ranker is the strongest training-free score in this study (Section 7.1), is therefore among every learned engine’s inputs. Type-specific blocks extend the vector to 19–25 dimensions: code metrics and CQP for Applications, reverse-USES blast radius for Libraries, queue capacity for Brokers, publisher/subscriber counts and QoS criticality for Topics, and CPU and memory for Hosts. PageRank, Reverse PageRank, betweenness and eigenvector centrality are computed on the QoS-weighted projection. They therefore carry QoS information into every predictor, including the “QoS-off” arms, which lack only the explicit QoS edge channel and QoS node columns. Because these topological summaries are available to any scorer, closed-form engines are genuine competitors rather than strawmen. CDI, at $O(|V|^2 + |V||E|)$, dominates analysis cost (Section 7.4).

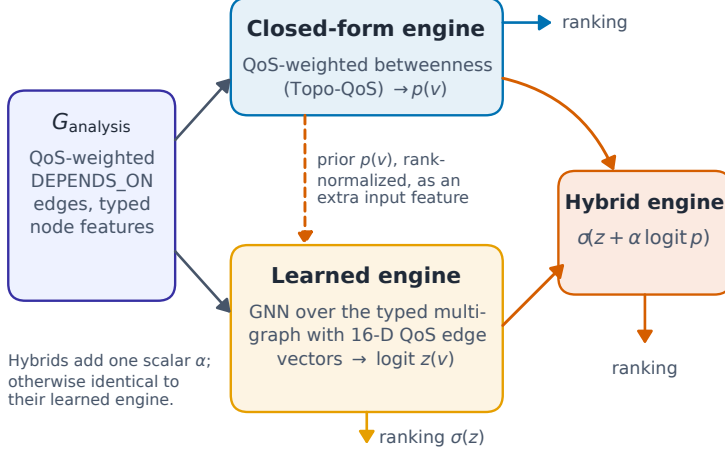
4. Ranking Engines and Ground Truth

SaG ranks components with three kinds of engine. The **closed-form engine** Topo-QoS is QoS-weighted betweenness on the dependency projection, the training-free engine registered before the study (Section 6.2); training-free dependency counts on the same projection are evaluated beside it (Section 6.2). The **learned engines** are graph neural networks trained on the typed multigraph (this section); only the typed engine propagates information into the Application nodes it scores (Section 6.2). The **hybrid engines** are learned engines that correct the closed-form score (Section 7.1.1). All are trained or scored against simulation oracles that run on a separate graph view (Sections 4.3–4.4). Figure 3 shows how the three engines relate and how they are evaluated. Full hyperparameters and training commands are on the experiment pages of the replication repository (Section 6.1).

4.1. Heterogeneous Graph Transformer

The learned engine HGT-QoS is a three-layer Heterogeneous Graph Transformer (HGT) [79] in PyTorch Geometric [88], with hidden dimension $D = 64$ and $H = 4$ heads. Entity-specific projections map raw features $x_v \in \mathbb{R}^{19-25}$ (Section 3.4) into the hidden space, $h_v^{(0)} = \text{LayerNorm}(\text{GELU}(W_{\tau(v)}x_v))$. For each meta-relation $\langle \tau(u), \phi(e), \tau(v) \rangle$, attention uses type-parameterized keys $K(u) = h_u W_K^{\tau(u)}$, queries $Q(v) = \tilde{h}_v W_Q^{\tau(v)}$ and values $V(u) = h_u W_V^{\tau(u)}$, scaled by a learned per-meta-relation prior $\mu_{\langle \tau(u), \phi(e), \tau(v) \rangle}$. That prior is the relation-typing mechanism that RQ2 tests. Its input is the full typed multigraph of G_{analysis} : all five entity types and seven edge types, the six structural relations of Table 1 together with the derived **DEPENDS_ON** edges. Message passing runs over this graph and its transpose, to capture downstream starvation and upstream backpressure, with residual connections, dropout 0.10 and layer normalization.

(a) Three ranking engines, one analysis graph



(b) Ground truth and evaluation

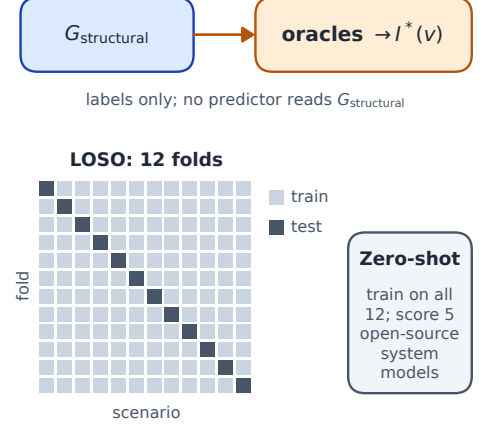


Figure 3: (a) SaG’s three ranking engines read the same analysis graph. The closed-form engine scores QoS-weighted betweenness $p(v)$; the learned engine outputs a logit $z(v)$. A hybrid engine gives the learned engine $p(v)$ as an extra input feature and adds a learned correction to it on the logit scale, $\sigma(z + \alpha \text{logit } p)$, with one learnable scalar α . (b) Ground truth comes from simulation oracles on the structural graph, which no predictor reads. Engines are evaluated by leave-one-scenario-out cross-validation over twelve synthetic architectures (each row trains on eleven and tests on the held-out one) and zero-shot on five open-source system models.

4.1.1. QoS Edge Encoding (16-D)

Each directed edge carries a vector $e_{uv} \in \mathbb{R}^{16}$. Index 0 is the coupling weight $w_E(e)$ (Section 3.2), index 1 the normalized count of simple paths through e , indices 2–8 a one-hot of the seven relation types, and indices 9–15 the middleware QoS parameters on PUBLISHES_TO/SUBSCRIBES_TO edges, zero elsewhere. Six QoS dimensions are active in our corpus: reliability, durability, priority, a flag for edges whose QoS departs from the scenario’s modal profile, and a deadline pair (active flag and log-deadline, populated on 463 of 615 topics). A seventh, max-blocking time, is reserved for hard real-time profiles and is zero throughout. The encoding is projected and added to the target representation before attention, $\hat{h}_v = h_v + W_{\text{edge}} e_{uv}$.

4.2. Prediction Head and Training Objective

A composite head predicts cascade impact, $\hat{I}^*(v) = \sigma(\text{MLP}_C(h_v \parallel \hat{a}_1(v) \parallel \hat{a}_2(v)))$. The two auxiliary heads $\hat{a}_1, \hat{a}_2$ act only as learned feature enrichment: $\hat{a}_1$ is supervised on I^* and $\hat{a}_2$ is unsupervised. The optimized objective combines regression with listwise and pairwise ranking:

$$\mathcal{L} = \text{MSE}(\hat{I}^*, I^*) + 0.5 \cdot \text{MSE}(\hat{a}_1, I^*) + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}}, \quad (5)$$

where $\mathcal{L}_{\text{rank}}$ is ListMLE [89] over the ground-truth permutation π ,

$$\mathcal{L}_{\text{rank}} = -\frac{1}{N} \sum_{i=1}^N \left(\hat{s}_{\pi_i} - \log \sum_{j=i}^N \exp(\hat{s}_{\pi_j}) \right), \quad (6)$$

and $\mathcal{L}_{\text{pairwise}}$ is a margin-ranking loss ($\gamma = 0.05$) over pairs whose true impacts differ by more than γ . A general form with a maintainability term and a consistency term tying the heads to the explanation layer exists but is switched off throughout, so the learned engines and the explanation layer share no parameters (Supplementary Section S20).

Models are trained with AdamW ($\eta = 3 \times 10^{-4}$, weight decay 10^{-4}) under cosine warm restarts for up to 300 epochs, with early stopping at patience 30 on an inner validation split. Five seeds {42, 123, 456, 789, 2024} are used throughout. Architectural hyperparameters follow conventional HGT values, and the loss coefficients were set by judgment. Neither was tuned on any evaluation split, so every learned arm is compared untuned.

4.3. Ground-Truth Simulation Oracles

Ground truth comes from failure simulations over the raw structural multigraph $G_{\text{structural}}$. Table 4 summarizes the three oracles this study uses.

Table 4: Simulation oracles, operational constructs, and evaluation roles.

Oracle	Physical Mechanism	Nature	Role in Evaluation
$I^*(v)$	Threshold cascade over topic feed loss + QoS ladder	Stochastic, 5 seeds	Primary ranking target (RQ1–RQ3)
$I_{\text{comp}}(v)$	Severity mixture: reachability + fragmentation	Deterministic	Explanation layer / Validate gate
$I_{\text{dyn}}(v)$	Discrete-event SimPy message queuing	Stochastic	Convergent-validity probe

Primary target, $I^*(v)$. The oracle crashes component v and propagates the outage in waves until a fixpoint. In each wave, (i) every dependent of a failed library fails outright, and (ii) every topic’s feed loss is recomputed as the larger of the rate-weighted share of its publishers that have failed and the share of its routing brokers that have failed. A subscriber’s loss is the mean feed loss over its topics, scaled by a declared QoS severity ladder ($\times 1.2$ RELIABLE, $\times 1.15$ high priority, $\times 1.05$ medium) and clamped to $[0, 1]$. A subscriber whose loss reaches the propagation threshold $\theta = 0.2$ fails with probability $\min(1, \text{loss}/\theta) \cdot d_k$, where the damping $d_k = \max(0.25, 1 - 0.15k)$ decreases with wave k . $I^*(v)$ is the mean continuous loss over the intact graph’s subscribers, averaged over five seeds that drive the Bernoulli draws and the propagation order. The ladder encodes severity rather than likelihood: a consumer that subscribes with reliable delivery or high priority is one that cannot tolerate missing samples, so the same loss harms it more.

Robustness to the oracle’s free parameters. The shipped labels are stable under the oracle’s own parameters. Re-labelling over $\theta \in \{0.1, 0.2, 0.3\}$ and damping steps $\{0.10, 0.15, 0.20\}$ keeps Application rankings at mean Spearman 0.86–0.99 against the shipped setting, and moves the closed-form engine’s accuracy only within 0.50–0.58 (Supplementary Table S34).

How much QoS is in this label. The ladder reads reliability and priority only, and disabling QoS scaling entirely leaves the Application ordering nearly intact: mean Spearman $\rho = 0.965$ against the ladder across the twelve folds (range 0.891–0.999); substituting a durability-aware $w(t)$ scaling moves it less still ($\rho = 0.977$). The top- K set is the sensitive construct: ladder and topology-only labels agree at mean Jaccard 0.678, so QoS changes *which* components are named critical rather than their order. I^* is therefore a near-topological target, which bounds what any QoS encoding can be credited with on it (Sections 7.1.2 and 7.2).

Further oracles. $I_{\text{comp}}(v)$ is a severity-weighted mixture of reachability loss, fragmentation, throughput loss and flow disruption, with unswept AHP coefficients (0.35, 0.25, 0.25, 0.15). It labels the explanation layer’s evaluation and is never used for forecasting. $I_{\text{dyn}}(v)$ is a SimPy [90] message-flow simulation of emission rates, stochastic latencies and broker buffer saturation. It returns the drop in delivered message rate to surviving consumers and serves as an independent convergent-validity probe. It agrees with I^* at $\rho = 0.627$, which is substantial but below I^* ’s own seed-to-seed test–retest of 0.811–1.000, so the two measure related but distinct constructs (Supplementary Section S9). Topic criticality is a predictor input, so it is masked out of every oracle’s severity term. Results established against one oracle are never transferred to another.

4.4. Input–Label Independence Guarantee

Features are built only from G_{analysis} : static topology, code metrics and declared QoS. Labels are computed only on $G_{\text{structural}}$ by the simulation oracles. No simulation output or runtime telemetry is exposed as a predictor input, and a CI test enforces the separation (`tests/test_independence_guarantee.py`). The guarantee is procedural: it rules out circular feature construction, not shared semantics. The oracle re-derives the same library and publish–subscribe dependency rules (Rules 1 and 5) from raw edges for its propagation step, and $I^*(v)$ is a reachability functional of the same architecture the predictors read. Ranking I^* is therefore the task of recovering, from the dependency projection, how far each component’s failure reaches, and simple counts over that projection are the natural first reference (Section 6.2).

5. The Explanation Layer: Standards-Grounded Criticality Attribution

A ranking says where risk is highest, not how to reduce it. A component may be critical because it is an unreplicated single point of failure, an error-propagating cascade hub, or a high-coupling maintainability

bottleneck. Each of these calls for a different intervention: replication, circuit breakers, or decoupling. The explanation layer attributes these causes after ranking. It reads the same node properties (Section 3.4), shares no parameters with the engines, and is not used as a ranker. We present it as a design proposal: its attributions are traceable to named metrics and standard sub-characteristics, but they have not yet been validated against developer judgement or against the outcome of the repairs they recommend (Section 8.3).

5.1. Grounding in ISO/IEC Standards

Following ISO/IEC 25010:2023 [64] and ISO/IEC 25019:2023 [65], criticality is profiled along **Reliability** (R), split into **Fault Tolerance** (FT) and **Availability** (A), and **Maintainability** (M). FT captures error-cascade potential and informs circuit breakers and redundancy. A captures structural single points of failure and informs replication. M captures coupling and code-level complexity and informs decoupling and refactoring. Safety and security, which need hazard logs, are out of scope.

5.2. Composite Quality Score

Figure 4 summarizes the layer. All metrics are rank-normalized to $[0, 1]$ within the graph and combined with AHP-derived weights [68]:

- $FT(v) = 0.45 \cdot \text{RPR}(v) + 0.30 \cdot \text{Deg}_{\text{in}}(v) + 0.25 \cdot \text{CDPot}_{\text{enh}}(v)$, over Reverse PageRank, normalized in-degree and normalized cascade depth on G_{analysis}^T ;
- $A(v) = 0.25 \cdot \text{AP}_c^{\text{dir}}(v) + 0.20 \cdot \text{QSPOF}(v) + 0.20 \cdot \text{BR}(v) + 0.25 \cdot \text{CDI}(v) + 0.10 \cdot w(v)$, over directed articulation severity, QoS-weighted SPOF severity, bridge ratio, the Connectivity Degradation Index and the node QoS weight;
- $R(v) = r_{\text{FT}} \cdot FT(v) + (1 - r_{\text{FT}}) \cdot A(v)$ with $r_{\text{FT}} = 0.36$;
- $M(v) = 0.35 \cdot \text{BT}(v) + 0.30 \cdot w_{\text{out}}(v) + 0.15 \cdot \text{CQP}(v) + 0.12 \cdot \text{CouplingRisk}_{\text{enh}}(v) + 0.08 \cdot (1 - \text{CC}(v))$, over betweenness, QoS-weighted efferent coupling, code-quality penalty, coupling risk and clustering.

The composite is $Q(v) = 0.80 \cdot R(v) + 0.20 \cdot M(v)$, and an ISO/IEC 25019 context-of-use vector can reweight R and M . Intra-dimension weights are shrunk towards a uniform prior ($\lambda = 0.70$). The elicited weights are not predictive: used as a ranker, $Q(v)$ scores $\rho = 0.205$ against I^* under LOSO, below every centrality baseline, and moving from the elicited towards a uniform prior improves it ($0.200 \rightarrow 0.319$; Supplementary Sections S1 and S25). $Q(v)$ is therefore used only to name the dimension along which a flagged component is weak, never to decide which components are flagged. The AHP matrices and their consistency diagnostics are in Supplementary Section S4. Within the explanation layer, components above the Tukey upper fence of Q are marked CRITICAL (mean 4.2% of components); in the pipeline, the components profiled are the ranking’s top- K . High A with low FT indicates a single point of failure that needs replication, while high FT indicates a cascade hub that needs circuit breakers (example card: Supplementary Section S19).

5.3. Counterfactual Verification of Remediation

The replication package includes tooling that generates candidate repairs (broker replication, circuit-breaker insertion, topic decoupling) for flagged components and verifies them counterfactually in memory: a repair is accepted only if it reduces systemic impact beyond seed noise ($\Delta \bar{I}_{\text{comp}} > \kappa \cdot \sigma_{\text{seed}}$, $\kappa \geq 1$) without introducing new articulation points. This paper reports no result from it. Comparing the repair class the layer recommends against alternative repairs under this verifier is the most direct test of the attributions, and is future work together with a developer study.

6. Experimental Setup

6.1. Corpus and Replication Package

The corpus comprises 2,812 components across seventeen architectures (Table 5). Twelve synthetic topologies form the LOSO folds. They span autonomous vehicles, financial trading, healthcare, industrial SCADA, smart-city IoT, telecom RAN, logistics, gaming, microservices, enterprise integration and air-traffic

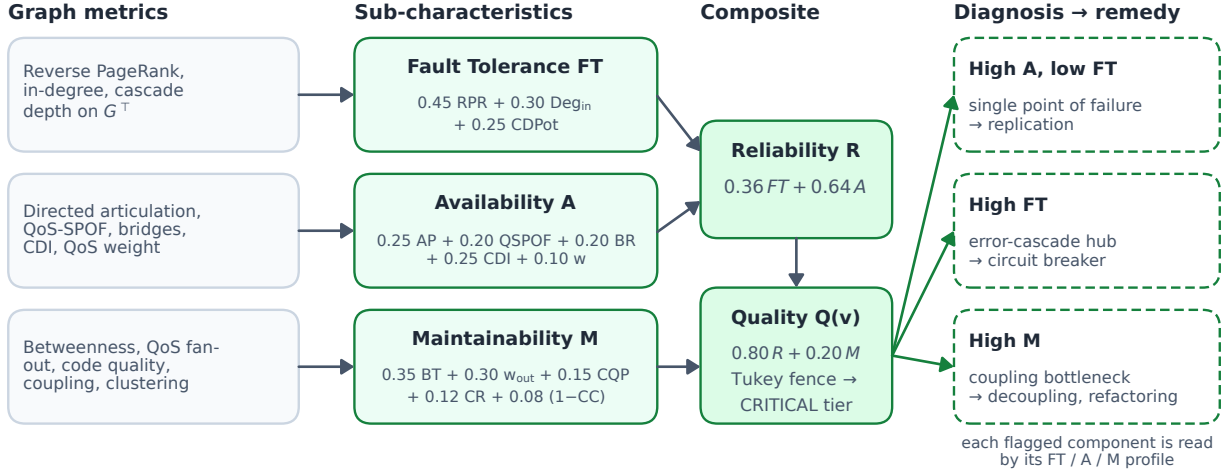


Figure 4: The explanation layer. Rank-normalized graph metrics feed the ISO/IEC 25010 sub-characteristics Fault Tolerance, Availability and Maintainability (CR: coupling risk; CC: clustering coefficient), which combine into Reliability and the composite $Q(v)$. A component above the Tukey fence of Q is flagged, and its $FT/A/M$ profile names the remediation class.

management. All twelve come from one generator family, as do their code metrics, so LOSO measures transfer across configurations of that generator. Each regenerates byte-identically from a committed configuration, and CI verifies this against a SHA-256 manifest. The generator couples QoS to topology by design: gateways and controllers preferentially attach to reliable, high-priority topics, sensors to best-effort ones, and operational criticality and hot-standby redundancy are assigned from a mix of topic QoS, application type and degree. A switch that removes this coupling (`qos_affinity: false`) regenerates a control corpus for Section 7.1.2. ATM is listed separately in Table 5 only because it also serves as the paper’s worked case study; it is an ordinary LOSO fold.

Table 5: Evaluation corpus. The twelve synthetic topologies are the LOSO folds; the five open-source system models are excluded from all training and used only for zero-shot transfer (Section 7.3). Counts are read from the committed topology files and verified in CI; per-scenario composition: Supplementary Section S13.

Dataset	$ V $	$ V_{app} $	Topics	Brokers	Hosts	Libs	$ E $
Synthetic evaluation scenarios (11)	2,387	1,295	588	60	194	250	10,657
Synthetic case study — ATM (1)	74	26	27	5	8	8	261
Synthetic subtotal (12 LOSO folds)	2,461	1,321	615	65	202	258	10,918
Open-source system models (5)	351	141	120	16	32	42	700
Total	2,812	1,462	735	81	234	300	11,618

The five open-source systems are hand-authored models. Autoware.universe (ROS 2), EdgeX Foundry, Home Assistant, and two meshes modelled after Online Boutique and Train-Ticket were each written by one author as typed multigraphs from public documentation (`saag/adapters/realworld_adapter.py`). They are not mechanical extractions. Brokers, QoS profiles, code metrics and host specifications are partly assumed. Two models depart materially from their originals: the Online Boutique model is a 22-application pub-sub mesh with four brokers, whereas the original is about eleven gRPC services with no broker, and the Train-Ticket model represents its service-discovery server as a broker. Neither contains a synchronous call edge. RQ3 therefore tests transfer to independently authored architecture models, not to deployed systems. The model topologies were last changed on 2026-09-06 and their attributes on 2026-09-10 (commits 2294f37 and bebc2ad), before any zero-shot result reported here.

Replication package. Datasets, harnesses, checkpoints and result artifacts are archived on Zenodo (see Data Availability). The public repository documents each experiment: its protocol, hyperparameters, `make` target, artifacts and the supplementary section holding its extended results (<https://github.com/onuralpyigit/software-as-a-graph/tree/jss-submission-v4/docs/research/jss/experiments>).

6.2. Predictors

SaG’s closed-form engine (**Topo-QoS**), learned engines (**HGT-QoS**, **GAT-QoS**) and hybrid engines are compared against the registered centrality baseline (**Topo**), untyped GNNs, and training-free dependency counts on the projection (Table 6). On a GNN, **-QoS** means the 16-D QoS edge vector (Section 4.1.1), and **Hybrid-X** is engine X corrected by the closed-form prior. **GAT** and **GAT-QoS** are untyped GATs matched to HGT in parameter budget, and **GAT-QoS** reads the same 16-D edge vector as **HGT-QoS**, so the four learned models form a 2×2 over typing (GAT vs. HGT) and the QoS channel. Smaller and projection-based variants that the study also ran are listed in Supplementary Section S29. All learned predictors ingest the native typed multigraph under LOSO. In QoS-off arms every edge weight is 1 and the QoS node columns are zeroed, though four centralities still carry QoS (Section 3.4).

What each learned engine can see. Every relation of the native multigraph points away from Applications (to Topics, Nodes and Libraries), and the untyped GATs aggregate along edge direction only. In **GAT**, **GAT-QoS** and **Hybrid-GAT** an Application therefore receives no messages, and its score is a function of its own feature vector: on the trained checkpoints, deleting every edge leaves every Application prediction unchanged. **HGT** reaches Applications through its reverse-direction pass, so an Application’s prediction depends on 35–59% of its graph; without that pass, as in the registered directionality control **HGT-QoS-U**, **HGT** scores Applications per node as well. We report the untyped arms as registered and use this property as a control (Section 7.2). Alongside them we report the feature-only control declared in Amendment 3, **GBM-Feat**: gradient boosting (scikit-learn defaults, one model per entity type) on the identical feature tensors, trained on the same graphs and per-graph label transform, with no graph model at all.

Table 6: Predictors reported in this paper. **-QoS**: QoS-weighted distances (Topo) or the 16-D QoS edge vector (GNNs); **Hybrid-X**: engine X corrected by the **Topo-QoS** prior. Every predictor run in the study, with the earlier names used by the registered plan and the artifacts: Supplementary Section S29.

Predictor	Evaluation Substrate	Typing	Edge Features	Parameters	Trained?	Empirical Role
<i>Training-free</i>						
Topo	G_{analysis} (app layer)	No	Dependency weight	0	No	Registered centrality baseline
Topo-QoS	App-Lib projection	No	Scalar $w(e)$	0	No	SaG closed-form engine
Betweenness (proj.)	App-Lib projection	No	None	0	No	Unweighted reference
InDeg	App-Lib projection	No	None	0	No	Direct dependents
Reach / Reach-QoS	App-Lib projection	No	None / $w(e)$	0	No	Transitive dependents
CDI	App-Lib projection	No	None	0	No	Connectivity degradation
<i>Learned: typing $\times$ QoS channel at matched parameter budget</i>						
GAT	Native Multigraph	Homogeneous	None	437,496	Yes	Untyped, no QoS; per-node at Applications
GAT-QoS	Native Multigraph	Homogeneous	16-D QoS Vector	429,992	Yes	Untyped SaG learned engine; per-node at Applications
HGT	Native Multigraph	Heterogeneous	Relation 1-hot; $w(e)=1$	434,620	Yes	Typed, no QoS channel
HGT-QoS	Native Multigraph	Heterogeneous	16-D QoS Vector	434,620	Yes	Typed SaG learned engine
<i>Hybrid engines: a learned engine corrected by the Topo-QoS prior</i>						
Hybrid-HGT	Native Multigraph	Heterogeneous	16-D QoS Vector	434,941	Yes	HGT-QoS + prior
Hybrid-GAT	Native Multigraph	Homogeneous	16-D QoS Vector	431,433	Yes	GAT-QoS + prior
<i>Attribution and directionality controls (Section 7.2)</i>						
GBM-Feat	Node features only	Per type	None	100 trees/type	Yes	Feature-only control (Amendment 3)
GBM-Feat-QoS	Node features only	Per type	None	100 trees/type	Yes	GBM-Feat + QoS node columns
GAT-QoS-nf	Native Multigraph	Homogeneous	16-D QoS Vector	429,992	Yes	GAT-QoS without QoS node columns
HGT-QoS-U	Native Multigraph	Heterogeneous	16-D QoS Vector	330,895	Yes	HGT-QoS without reverse pass; per-node at Applications
GAT-w	Native Multigraph	Homogeneous	Scalar $w(e)$	439,272	Yes	Capacity control; per-node at Applications

Closed-form scores. **Topo-QoS** is QoS-weighted betweenness on the Application–Library **DEPENDS_ON** projection (Rules 1 and 5), with edge distances $d(e) = 1/(w(e) + 10^{-6})$ so that strongly coupled edges attract shortest paths; the projection is needed because on the raw multigraph messages route through topics and brokers and Application betweenness vanishes. The registered baseline **Topo** is betweenness as the analysis stage computes it on the application layer of G_{analysis} , over dependency-weight distances. Both were specified with an articulation term that reads zero in the evaluated implementation, so they rank exactly as the betweenness scores just described; restoring it lowers both (Supplementary Section S22). Because **Topo** and **Topo-QoS** differ in substrate as well as in QoS weighting, we add unweighted betweenness on the same projection as a like-for-like reference.

Dependency counts. Amendment 7 registered four further training-free rankers on the same projection, directed dependent $\rightarrow$ dependency: **InDeg**, the number of direct dependents of v ; **Reach**, the number of its transitive dependents; **Reach-QoS**, the sum over transitive dependents of the best-path product of edge weights; and **CDI**, the Connectivity Degradation Index alone. They answer the question the oracle asks, *how*

many components depend on v , in the most direct way, and cost a single graph traversal. No predictor reads $G_{\text{structural}}$ (Section 4.4).

6.3. Metrics, Protocols and Statistics

Population. Every predictor in a table is scored on the same node population, the Application set V_{app} . Pooling entity types conflates distinct base rates. Against I_{comp} , RM correlates at $\rho = 0.597$ on Applications but only 0.217 pooled over all types (Supplementary Section S6).

Metrics. Ranking is measured by Spearman ρ against $I^*(v)$ with mid-ranks for ties. Critical-set identification is measured by $\text{Overlap}@K$, the fraction of the true top- K recovered by the predicted top- K with $K = \text{round}(0.20 |V_{\text{app}}|)$, at which top- K precision, recall and F_1 coincide; ties at the K -th position are broken by node identifier on both sides. Because 21–52% of Applications per fold have zero simulated impact, we also report $\rho_{>0}$ on the active stratum, which isolates how well a ranker orders the components whose failure reaches anyone. PR-AUC, $F_1@7$ and nDCG@10 are reported in Supplementary Section S15.

Protocols. Under *LOSO*, models train on eleven scenarios and are tested zero-shot on the twelfth, over all 12 folds and five seeds, with equal 3-layer depth and inner-split early stopping. Under *zero-shot transfer*, models trained on all twelve scenarios are evaluated without fine-tuning on the five system models. In-distribution node-split results are in Supplementary Section S17.

Statistics. We use paired Wilcoxon signed-rank tests over folds [91, 92] and bootstrap 95% CIs ($B = 2,000$) over folds [93]; with 12 folds, the smallest attainable two-sided p is 0.00049. Folds share ten of eleven training scenarios, so the tests are anti-conservative [94, 95], and we read all p -values as nominal, next to fold-level sign counts and intervals. A non-significant difference is not read as equivalence [96], and a significant contrast against one reference beside a non-significant one against another is not read as a difference between the two [97].

6.4. Analysis Plan and Deviations

Registered analysis plan. Before the twelve-fold harness produced any result, we registered the primary contrast, HGT-QoS vs. Topo-QoS, together with HGT vs. Topo-QoS and Holm correction across the two. We call it *registered* rather than pre-registered, because the plan is a file in our repository with no third-party timestamp. Four amendments registered further contrasts, each before its run and each Holm-corrected within its own family: the matched 2×2 (Amendment 2), Hybrid-HGT (Amendment 5), Hybrid-GAT (Amendment 6) and the training-free dependency counts with the QoS-attribution controls (Amendment 7). Every other contrast is exploratory, including the attribution controls of Section 7.2 (Amendment 8, written after their results). Because the sequence was adaptive (Amendment 6 followed Amendment 2’s result), we also pool all thirteen registered contrasts under one Holm correction (`reproduce/omnibus_holm.py`). Both hybrid primaries remain significant (Hybrid-GAT $p_{\text{omni}} = 0.019$, Hybrid-HGT $p_{\text{omni}} = 0.041$), and no other registered contrast reaches $\alpha = 0.05$ ($p_{\text{omni}} \geq 0.46$). The Amendment 7 contrasts form their own family, declared with its own decision rules and Holm correction, and are not pooled. Supplementary Section S24 lists all eight amendments with dates and outcomes.

How the hybrids came about. The hybrids were registered before any hybrid result existed, but their design was proposed after the per-fold pattern of HGT-QoS against Topo-QoS was known (Amendments 5 and 6, 2026-09-23/24). Their registration controls knowledge of the outcome, not of which design to try, so we treat them as confirmed on their registered contrasts and as hypothesis-generating beyond them.

Registered arms not run. One arm registered in Amendment 2 was not run: a label-side sweep with the oracle’s QoS ladder disabled, whose question Section 4.3 answers by label agreement instead. The capacity-only control (GAT-w) and the directionality control (HGT-QoS-U) of the same amendment were run, as was the gradient-boosting ranker on the learned engines’ node features declared in Amendment 3 (GBM-Feat); Section 7.2 reports all three. Every conclusion below is scoped to the arms that ran.

Which sweep each number comes from. Learned cells move across code revisions and devices (Section 8.2), so each table draws every learned value from one sweep: Table 7 and the matched control from the CPU sweeps of Amendments 2, 5 and 6, and the registered primary contrast from the GPU sweep the plan specified (Supplementary Section S30). Training-free values are identical across sweeps and devices. The Amendment 7

rankers were scored by a separate harness that first reproduced every published per-fold Topo-QoS value to three decimals.

7. Results

All results are reported on the Application population (V_{app}) against the primary oracle $I^*(v)$, under the input-label independence guarantee (Section 4.4). Per-fold results, secondary strata and extended protocol notes are in the Supplementary Material and the experiment pages of the replication repository (Section 6.1). Figure 5 summarizes the main findings.

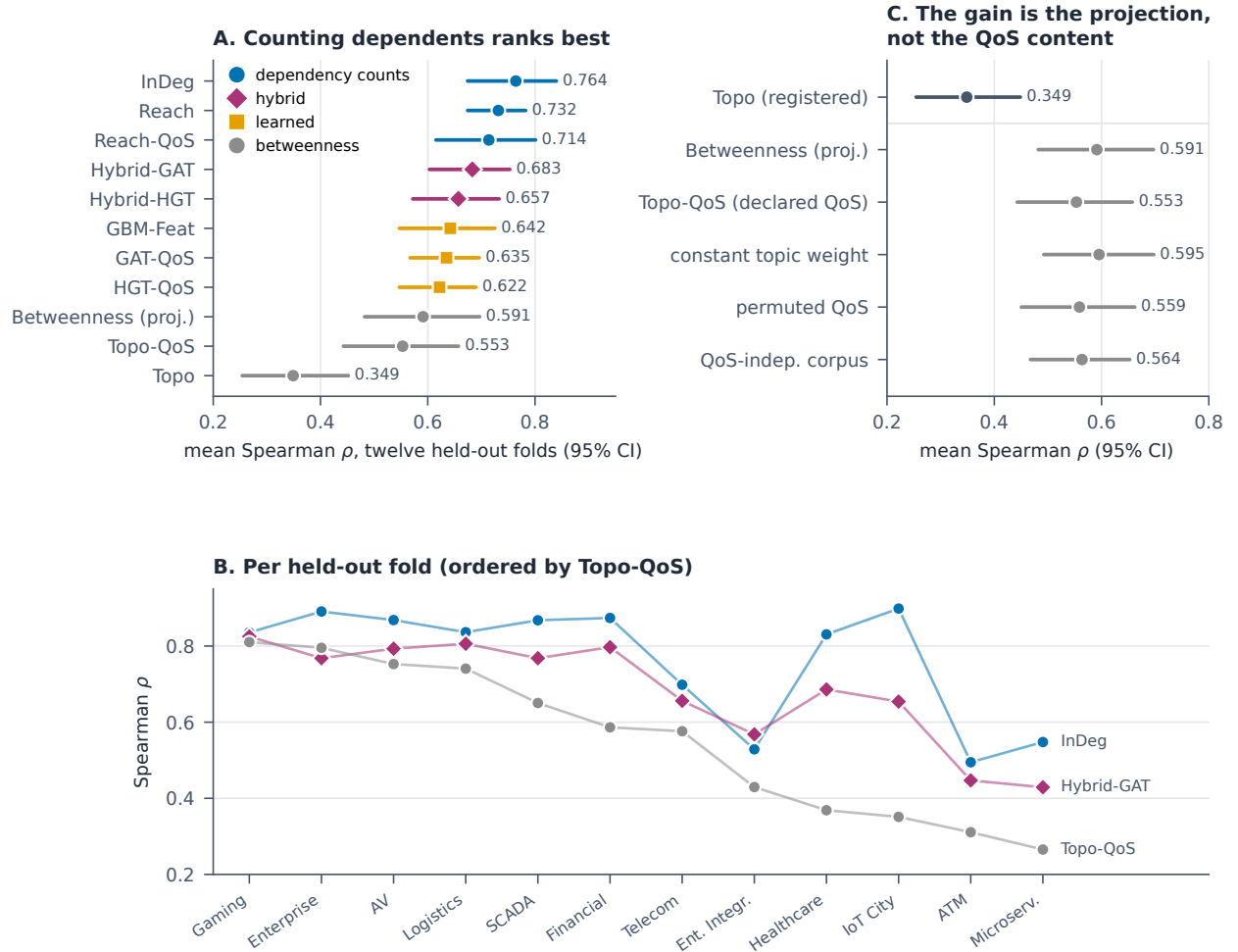


Figure 5: Main results at a glance, Application population, twelve LOSO folds. (A) Mean Spearman ρ with 95% bootstrap intervals for every engine: dependency counts on the projection (InDeg, Reach) rank best, ahead of the hybrid, learned, feature-only and betweenness-based engines. (B) Per held-out fold, InDeg, the best hybrid (Hybrid-GAT) and the registered closed-form engine (Topo-QoS); folds ordered by Topo-QoS. (C) Where the closed-form gain comes from: the registered baseline Topo, unweighted betweenness on the projection, Topo-QoS, and Topo-QoS with a constant topic weight (Mult) or with QoS profiles permuted across topics (Perm).

7.1. RQ1: Ranking Cascading-Failure Impact on Unseen Architectures

Finding 1. On SaG’s dependency projection, counting a component’s dependents ranks its cascading-failure impact best. InDeg reaches $\rho = 0.764$ and Reach 0.732 on held-out architectures, above the registered closed-form engine on 12 and 11 of 12 folds, and above every learned and hybrid engine, including a regressor

trained on SaG’s per-component features. Among the engines that combine structural cues, the two hybrids are the only ones that significantly beat the closed-form engine (+0.103 and +0.130, 11/12 folds, registered).

Each of the twelve folds holds out one scenario and trains on the remaining eleven, and every predictor is scored on the same Application node set (26 to 300 nodes, K between 5 and 60). Table 7 reports all engines; training-free values are identical across sweeps and devices.

Table 7: Main results under LOSO (twelve synthetic architectures, Application population; learned engines: five seeds, CPU sweeps) and on the five hand-authored system models. $\Delta\rho$ is paired by fold against the registered closed-form engine **Topo-QoS**, with a bootstrap 95% CI ($B = 2,000$) and a two-sided Wilcoxon test; p_{Holm} is given within each registered family (Amendments 5, 6 and 7). Against their own learned engines the hybrids gain +0.035 ($p = 0.73$) and +0.048 ($p = 0.30$). [†]Scored by the Amendment 7 harness on the same models and oracle settings; in that harness **Topo-QoS** scores 0.582 rather than 0.526 (Supplementary Table S32). Per-fold values: Supplementary Sections S23 and S31; the registered GPU sweep of **HGT-QoS**: Section S30.

Predictor	LOSO, twelve synthetic architectures					Five system models	
	Mean ρ [95% CI]	$\Delta\rho$ vs Topo-QoS [95% CI]	Won	p (p_{Holm})	Overlap@ K	ρ [95% CI]	PR-AUC
<i>Training-free</i>							
Topo	0.349 [0.254, 0.452]	−0.204 [−0.286, −0.122]	0/12	0.0005	0.366	0.511 [0.346, 0.703]	0.474
Topo-QoS	0.553 [0.443, 0.657]	—	—	—	0.388	0.526 [0.357, 0.699]	0.474
Betweenness (proj.)	0.591 [0.482, 0.697]	+0.038 [+0.014, +0.063]	9/12	0.0210	0.411	0.617 [†] [0.485, 0.785]	0.526 [†]
CDI	0.233 [0.085, 0.358]	−0.320 [−0.451, −0.189]	1/12	0.0034 (0.0068)	0.342	0.459 [†] [0.265, 0.657]	0.430 [†]
Reach-QoS	0.714 [0.615, 0.801]	+0.161 [+0.097, +0.235]	12/12	0.0005 (0.0020)	0.465	0.933 [†] [0.899, 0.968]	0.888 [†]
Reach	0.732 [0.674, 0.782]	+0.178 [+0.088, +0.268]	11/12	0.0034 (0.0068)	0.341	0.938[†] [0.879, 0.991]	0.933[†]
InDeg	0.764 [0.674, 0.840]	+0.211 [+0.132, +0.299]	12/12	0.0005 (0.0020)	0.506	0.863 [†] [0.734, 0.952]	0.752 [†]
<i>Learned</i>							
HGT-QoS	0.622 [0.547, 0.690]	+0.069 [−0.046, +0.174]	8/12	0.266	0.426	0.760 [0.714, 0.819]	0.713
GAT-QoS	0.635 [0.567, 0.696]	+0.082 [−0.046, +0.201]	7/12	0.233	0.438	0.805 [0.759, 0.868]	0.790
<i>Hybrid: learned engine corrected by the Topo-QoS prior</i>							
Hybrid-HGT	0.657 [0.572, 0.733]	+0.103 [+0.055, +0.152]	11/12	0.0034 (0.0068)	0.435	0.695 [0.643, 0.730]	0.602
Hybrid-GAT	0.683 [0.603, 0.753]	+0.130 [+0.075, +0.190]	11/12	0.0015 (0.0029)	0.450	0.662 [0.597, 0.727]	0.600

Counting dependents is the strongest ranker. InDeg, the number of components that directly depend on v in the projection, reaches $\rho = 0.764$ [0.674, 0.840] and the highest Overlap@ K (0.506). It beats **Topo-QoS** on all twelve folds (+0.211, Holm $p = 0.0020$), **HGT-QoS** and **GAT-QoS** on ten (+0.143 and +0.129, $p \leq 0.005$), both hybrids on ten and eleven (+0.108 and +0.081, $p = 0.0024$), and the feature-only regressor **GBM-Feat** of Section 7.2 on ten (+0.122, $p = 0.0024$; Supplementary Table S31). Transitive reach is close behind (0.732), and QoS-weighted reach keeps the most signal among components that actually propagate failure after InDeg ($\rho_{>0} = 0.399$ against 0.516 for InDeg and 0.280 for **Topo-QoS**). These rankers need no training, read only the projection, and cost at most 0.15 s per scenario (Section 7.4). The learned engines receive normalized in-degree among their inputs (Section 3.4) but do not reproduce its ranking on held-out architectures; on this oracle, what learning adds to the projection is not yet more than what a direct count of dependents captures. CDI alone, by contrast, ranks poorly (0.233): path-length inflation measures something other than how many components lose their feeds.

The projection also tells inert components from active ones. Between 21% and 52% of each held-out population carries zero simulated impact. The rule “a component propagates failure if and only if it has at least one dependent” identifies them with 94% balanced accuracy on the folds and 97% on the system models (Supplementary Table S35). Much of every full-population correlation reflects this separation. Restricted to active components, the betweenness, learned and hybrid engines keep roughly half of their correlation (Supplementary Section S25), while InDeg keeps two thirds (0.516), the most of any ranker.

Both hybrids significantly outperform the closed-form engine. Hybrid-HGT reaches $\rho = 0.657$ (+0.103, Holm $p = 0.0068$) and Hybrid-GAT reaches $\rho = 0.683$ (+0.130, CI [+0.075, +0.190], Holm $p = 0.0029$), each on 11 of 12 folds. Each meets the decision rule registered before its run. Both remain significant under one Holm correction pooled over all thirteen registered contrasts of the study ($p_{\text{omni}} = 0.041$ and 0.019; Section 6.4), and both also beat unweighted betweenness on the projection ($p = 0.043$ and 0.012). They are the only learned models that significantly beat **Topo-QoS**: the feature-only regressor of Section 7.2 does not (+0.089, $p = 0.176$). They do not significantly beat their own learned engines (+0.035, $p = 0.73$; +0.048, $p = 0.30$). Their advantage over **Topo-QoS** comes more from consistency than from a larger mean gain: anchoring on the prior narrows the fold-level spread of the difference (CI width 0.10 against 0.22 for **HGT-QoS**), because the hybrid’s errors covary with the prior it is compared against. On their own, **HGT-QoS** (+0.069,

$p = 0.266$) and GAT-QoS (+0.082, $p = 0.233$) lead Topo-QoS numerically, and the registered confirmatory contrast on the GPU sweep agrees (+0.085, $p = 0.151$, Holm 0.303; Supplementary Section S30).

Closed-form and learned engines are complementary. HGT-QoS loses to Topo-QoS on four folds, all where the closed-form engine is strongest: Real-Time Gaming, Enterprise (0.426 vs. 0.795), AV System and Telecom RAN (0.427 vs. 0.576). Its largest gains come where the closed-form engine is weakest: Healthcare, IoT Smart City, ATM and Microservices, +0.210 to +0.362. The prior removes the failure mode: on Enterprise, Hybrid-HGT and Hybrid-GAT reach 0.735 and 0.768, and Telecom RAN turns from a loss into a win for both. InDeg has no such failure mode on this corpus: it beats Topo-QoS on every fold, and its three weakest folds (ATM, Enterprise Integration and Microservices) are also the hybrids’ three weakest (Figure 5B).

Labels are reproducible. Re-running the oracle across five seeds gives a test–retest rank correlation of 0.811–1.000 (median 0.982), well above every engine’s mean ρ .

7.1.1. How the Hybrids Are Built

Each hybrid gives a learned engine the rank-normalized Topo-QoS score as one extra input per Application and Library, and adds a learned correction to that score on the logit scale, $\hat{I}^*(v) = \sigma(z(v) + \alpha \logit(p(v)))$, with one learnable α (Figure 3a). Everything else matches the underlying engine. Hybrid-HGT is built on HGT-QoS (Amendment 5; 321 extra parameters) and Hybrid-GAT on GAT-QoS (Amendment 6; 1,441 extra parameters). Each was registered with its contrasts and decision rule before any run, with no setting tuned, and evaluated in its own CPU sweep with its comparators re-run in the same invocation. On the five system models both stay above the betweenness scores (0.695 and 0.662 vs. 0.511–0.526) but below the pure learned engines (0.760 and 0.805; Section 7.3); by the rule registered in Amendment 6, Hybrid-GAT therefore does not replace Hybrid-HGT as the registered hybrid ($0.662 < 0.695$).

7.1.2. What Drives the Closed-Form Gain

Finding 2. The closed-form gain from 0.349 to 0.553 comes from reading the dependency projection, not from the content of the declared QoS contracts. On this oracle, SaG ranks as well without QoS profiles as with them, which lowers what a team has to declare before the analysis is useful.

The registered comparator Topo reads betweenness from the analysis stage’s application-layer graph, whereas Topo-QoS computes QoS-weighted betweenness on the Application–Library projection; the two differ in substrate as well as in weighting. Three controls registered in Amendment 7 separate the two (Supplementary Table S33). On the same projection, unweighted betweenness scores 0.591, slightly above Topo-QoS (+0.038, 9/12 folds). Fixing every topic weight to a constant, so that a dependency’s weight reflects only how many topics join the pair, scores 0.595; permuting QoS profiles across topics scores 0.559 (−0.006 against the declared profiles, 5/12, $p = 0.73$). On a control corpus regenerated with QoS no longer steering topology, QoS weighting again changes projection betweenness by −0.035. Decision rule R2 of Amendment 7 therefore applies: the gain is attributed to the dependency projection, and specifically to topic multiplicity in Rule 1, not to the declared contracts. This matches the label analysis of Section 4.3, where the oracle’s ordering is 96.5% topological. Oracles that express deadline misses, durability replay or priority inversion are where QoS content can be expected to matter.

7.2. RQ2: What Learned Engines Need

Finding 3. Learned accuracy comes from SaG’s per-component features. At matched capacity, relation-typed parameters add nothing measurable (typing main effect −0.014), and neither does message passing: a gradient-boosted regressor that reads the same features with no graph model matches the learned engines ($\rho = 0.642$, against 0.635 for GAT-QoS and 0.622 for HGT-QoS). The untyped engine’s QoS gain (+0.072) is carried by three declared-coupling node features (+0.095, 11 of 12 folds), not by the 16-D edge channel (−0.023), and without its reverse pass HGT receives no messages at Applications and loses nothing ($\rho = 0.632$, $p = 0.91$).

A first comparison against small untyped GATs (28,168 parameters against HGT’s 434,620, reading at most a scalar edge weight) credited typing with a large gain (+0.234 without QoS). That gain is an effect of capacity and channel width (Supplementary Sections S26 and S30), and the control registered in

Amendment 2 removes both differences. **GAT** and **GAT-QoS** are untyped GATs at HGT’s parameter budget (437,496 and 429,992), and **GAT-QoS** reads the same 16-D edge vector as **HGT-QoS**, including the relation one-hot. All four matched arms ran in one CPU sweep, and the decision rule was fixed before any control result existed (Table 8).

Table 8: The 2×2 with capacity and edge-channel width matched (Amendment 2): **GAT** ($\neg T \neg Q$, 437,496 parameters), **HGT** ($T \neg Q$, 434,620), **GAT-QoS** ($\neg T Q$, 429,992), **HGT-QoS** ($T Q$, 434,620). One CPU sweep, twelve LOSO folds, five seeds, Application population. Holm correction across the three orthogonal quantities; simple effects are descriptive. Cell means: 0.563, 0.548, 0.635, 0.622. The Q factor switches the 16-D edge channel and the three QoS node columns together; Table 9 separates them.

Quantity	Contrast	$\Delta\rho$	95% CI	Won	W	p	p_{Holm}
<i>Three orthogonal quantities, Holm-corrected across these three</i>							
Typing (main effect)	averaged over Q	-0.014	[-0.052, +0.023]	4/12	29.0	0.470	0.940
QoS inputs (main effect)	averaged over T	+0.073	[+0.013, +0.120]	10/12	13.0	0.043	0.127
Typing $\times$ QoS interaction	difference of differences	+0.001	[-0.050, +0.042]	6/12	34.0	0.733	0.940
<i>Simple effects — descriptive, not separately corrected</i>							
Typing, QoS absent	HGT vs. GAT	-0.015	[-0.064, +0.033]	5/12	31.0	0.569	—
Typing, QoS present	HGT-QoS vs. GAT-QoS	-0.013	[-0.054, +0.026]	4/12	27.0	0.380	—
QoS inputs, typing absent	GAT-QoS vs. GAT	+0.072	[+0.028, +0.109]	10/12	9.0	0.016	—
QoS inputs, typing present	HGT-QoS vs. HGT	+0.073	[-0.002, +0.136]	10/12	19.0	0.129	—

Relation-typed parameters add nothing measurable. HGT and HGT-QoS are within 0.015 of their untyped counterparts and win only 4–5 of 12 folds against them; the typing main effect is -0.014 $[-0.052, +0.023]$. The interval does not establish equivalence at a margin of ± 0.05 , so the finding is that no benefit of typing is detectable at this corpus size. The untyped arms, however, receive no messages at the Applications they score (Section 6.2). The registered 2×2 therefore compares typed message passing with per-component learning, not two message-passing architectures, and Table 9 asks what each of its ingredients contributes.

Table 9: Attribution controls (exploratory; Amendment 8). One CPU sweep, twelve LOSO folds, five seeds, Application population, with **GAT** and **GAT-QoS** re-run in the same invocation (bit-identical to Table 8). **GAT-QoS-nf** is **GAT-QoS** reading **GAT**’s node features; each of its inputs is bit-identical to one parent arm. **GBM-Feat(-QoS)** is gradient boosting on **GAT**’s (**GAT-QoS**’s) node features. Holm correction across the five contrasts. Cell means: **GBM-Feat** 0.642, **GBM-Feat-QoS** 0.632, **GAT-QoS-nf** 0.540. Per-fold values: Supplementary Section S32.

Quantity	Contrast	$\Delta\rho$	95% CI	Won	W	p	p_{Holm}
QoS node columns, edge channel held	GAT-QoS vs. GAT-QoS-nf	+0.095	[+0.048, +0.135]	11/12	5.0	0.0049	0.024
QoS edge channel, node features held	GAT-QoS-nf vs. GAT	-0.023	[-0.046, -0.004]	3/12	15.0	0.064	0.192
QoS node columns, no graph model	GBM-Feat-QoS vs. GBM-Feat	-0.010	[-0.028, +0.006]	5/12	30.0	0.519	1.000
Neural vs. trees, QoS-off features	GAT vs. GBM-Feat	-0.079	[-0.134, -0.028]	1/12	7.0	0.0093	0.037
Neural vs. trees, QoS-on features	GAT-QoS vs. GBM-Feat-QoS	+0.003	[-0.048, +0.054]	7/12	39.0	1.000	1.000

The QoS gain is carried by three node features, not the edge channel. The Q factor of Table 8 switches two inputs together: the 16-D edge vector and three node columns holding each component’s declared coupling weights (w , w_{in} , w_{out} ; Section 3.4). Removing the three columns from **GAT-QoS** costs 0.095 (11 of 12 folds, Holm $p = 0.024$), while adding the edge channel to **GAT** changes nothing (-0.023 , 3 of 12). The training stabilization the channel appeared to provide follows the same columns: the median within-fold seed spread is 0.010 for **GAT-QoS**, 0.083 for **GAT** and 0.136 for **GAT-QoS-nf**. The architecture explains this: no edge reaches an Application in the untyped engines, so the channel can affect Application scores only indirectly, through weights shared with other entity types during training.

Message passing adds nothing measurable on this target. The untyped engines score each Application from its own features, and the typed HGT, which aggregates over 35–59% of the graph, does not outperform them. The directionality control registered in Amendment 2, run after Amendment 8, tests this within HGT: removing the reverse pass removes every message into an Application, and **HGT-QoS-U** (330,895 parameters) reaches $\rho = 0.632$ $[0.570, 0.687]$ against 0.622 for **HGT-QoS** (-0.010 for **HGT-QoS**, 6 of 12 folds, $p = 0.91$; Supplementary Section S32). Its median seed spread also falls, from 0.056 to 0.020. The capacity control registered with it answers from the untyped side: **GAT-w**, an untyped GAT at HGT’s parameter budget (439,272) reading the same QoS node features and a scalar edge weight, reaches 0.633 (-0.011 for **HGT-QoS**, 5 of 12 folds, $p = 0.68$) and is level with **GAT-QoS** (-0.002), so the width of an edge channel that never

reaches an Application is immaterial. The feature-only regressor, **GBM-Feat** (declared in Amendment 3), reads the identical feature tensors with no graph model and reaches $\rho = 0.642$ [0.547, 0.725]. That is on par with **GAT-QoS** (+0.003 for the matched-feature pair) and with **HGT-QoS** (+0.021, 7 of 12 folds, paired across sweeps whose shared arms are bit-identical). The three QoS columns do not help the regressor (−0.010). The neural model needs them to reach what the trees extract from the QoS-weighted centralities: without them, **GAT** trails **GBM-Feat** on 11 of 12 folds (−0.079). The learned engines’ margin over closed-form ranking is therefore the learned combination of SaG’s per-component features, not aggregation over the graph. Only the hybrids, which add the closed-form score to those features, significantly beat closed-form ranking, and no learned model reaches a direct count of dependents (Section 7.1).

How much the target can reward. $I^*(v)$ is a near-topological target: a topology-only relabeling recovers its ordering at mean $\rho = 0.965$, and QoS acts mainly at its top- K boundary (Section 4.3). The learned models reach QoS through the QoS-weighted centralities and declared coupling weights among their node features, and on this target the edge encodings contribute nothing measurable. Two things are needed before either the encodings or message passing can be credited with contract semantics: oracles that express deadline misses, durability replay or priority inversion, and a substrate on which messages reach every scored component.

7.3. RQ3: Zero-Shot Transfer to Models of Open-Source Systems

Finding 4. Trained only on synthetic scenarios, the learned models transfer to five hand-authored models of open-source systems at $\rho = 0.757$ –0.831 and nearly double the top- K overlap of betweenness-based scores. The feature-only regressor reaches 0.757, so most of that margin comes from learning on SaG’s per-component features, not from message passing or QoS inputs. Dependency counts on the models’ projections transfer without any training (0.86–0.94). With five models written by one author, this evidence is exploratory.

The five systems are hand-authored models of Autoware.universe (ROS 2), EdgeX Foundry and Home Assistant, plus meshes modelled after Online Boutique and Train-Ticket (Section 6.1). They were written independently of the scenario generator but are models rather than extractions, and they carry labels from the same oracles. **HGT-QoS**, **GAT-QoS**, **GAT** and **GBM-Feat** were trained on all twelve synthetic scenarios and evaluated zero-shot at the same 3-layer, 300-epoch budget as every other learned result. No system model contributed gradients or checkpoint selection. Where a system declares no QoS manifest, standard middleware defaults (ROS 2 Best-Effort/Reliable, MQTT QoS 0/1) are applied uniformly across all predictors.

Table 10: Zero-shot transfer to hand-authored models of five open-source systems: Spearman ρ against $I^*(v)$ on the Application population. Learned engines were trained on all twelve synthetic scenarios (3 layers, 300 epochs; five seeds, $\pm$ = spread over seeds). Training-free scores are deterministic and scored on identical labels and node sets. The last column restricts **HGT-QoS** to components with positive impact. **GBM-Feat**: no graph model (Section 6.2). All five models are encoded as publish-subscribe graphs; the grouping follows the paradigm of the *original* system. Identification metrics: Supplementary Table S14; bootstrap intervals and the active stratum for every predictor: Supplementary Section S27.

System model	$ V_{\text{app}} $	$n_{>0}$	Topo	Topo-QoS	HGT-QoS	GAT-QoS	GAT	GBM-Feat	HGT-QoS $\rho_{>0}$
<i>Originals are publish-subscribe systems</i>									
Autoware.universe (ROS 2)	32	19	0.307	0.378	0.716 $\pm$ 0.081	0.758 $\pm$ 0.019	0.778 $\pm$ 0.023	0.698 $\pm$ 0.009	+0.517
EdgeX Foundry (Industrial IoT)	22	10	0.534	0.534	0.793 $\pm$ 0.037	0.815 $\pm$ 0.055	0.853 $\pm$ 0.015	0.792 $\pm$ 0.005	+0.183
Home Assistant (Smart Home)	24	17	0.297	0.289	0.864 $\pm$ 0.063	0.925 $\pm$ 0.023	0.927 $\pm$ 0.024	0.736 $\pm$ 0.008	+0.702
<i>Originals are RPC systems (modelled as pub-sub)</i>									
Online Boutique (pub-sub model)	22	8	0.891	0.888	0.710 $\pm$ 0.070	0.750 $\pm$ 0.119	0.810 $\pm$ 0.008	0.747 $\pm$ 0.037	−0.031
Train-Ticket Booking Mesh	41	14	0.528	0.541	0.717 $\pm$ 0.096	0.777 $\pm$ 0.007	0.786 $\pm$ 0.013	0.810 $\pm$ 0.002	−0.192
Mean	—	—	0.511	0.526	0.760	0.805	0.831	0.757	+0.236

Learned models transfer, and neither typing, QoS inputs nor message passing is what transfers. Every learned model leads the betweenness-based scores on 4 of 5 systems (a sign test on five systems cannot reach significance, $p = 0.375$; intervals over five systems are descriptive). Among the learned models, each architectural addition lowers transfer slightly:

- plain **GAT**, which reads no QoS input and, like every untyped engine, scores Applications from their own features, transfers best ($\rho = 0.831$, Overlap@ K 0.548, PR-AUC 0.811);
- adding the edge channel gives 0.821, and adding the QoS node columns as well (**GAT-QoS**) gives 0.805;

- **HGT-QoS**, the only engine that propagates information into Applications, transfers no better than the feature-only regressor (0.760 against 0.757), and without its reverse pass (HGT-QoS-U) it transfers at 0.804, higher on all five systems.

With five systems these orderings are descriptive. The robust finding is the gap to betweenness-based ranking: top- K overlap averages 0.40–0.55 for the learned models against 0.248 for the betweenness scores, and PR-AUC 0.71–0.81 against 0.474–0.521. On EdgeX, symmetric adapter-to-broker stars create betweenness ties that collapse betweenness-based triage entirely (Overlap@ K = 0.000).

Dependency counts transfer without training. Scored on the same models and oracle settings, transitive reach reaches ρ = 0.938, QoS-weighted reach 0.933 and InDeg 0.863, with PR-AUC up to 0.933 and Overlap@ K up to 0.760 (Table 7; Supplementary Table S32). Reach also orders the active components well ($\rho_{>0}$ = 0.871), where the betweenness scores and **HGT-QoS** (+0.236, interval spanning zero) do not; among the other learned models, plain **GAT** reaches +0.377 [+0.078, +0.677], **GAT-QoS** +0.319 [+0.001, +0.638] and the feature-only regressor +0.156 [−0.066, +0.374], percentile intervals over five systems that carry no coverage guarantee. Part of why every ranker scores higher here than under LOSO is the label structure: the system models are smaller (28 vs. 110 Applications on average) and half of their Applications are inert (51% vs. 31%), so separating inert from active components carries more of the correlation (Supplementary Table S35). **Scope.** The models were written by one author, and two depart materially from their originals (Section 6.1). $\rho_{>0}$ for **HGT-QoS** is positive on the three models of publish–subscribe systems and non-positive on the two modelled after RPC systems; because both are encoded as publish–subscribe graphs, this split cannot be attributed to call-tree semantics. An earlier version of this study withdrew the transfer claim under a non-matched protocol; the results here come from the common 3-layer, 300-epoch protocol (Amendment 4), and an earlier 2-layer configuration leaves every conclusion unchanged (Supplementary Section S29).

7.4. RQ4: Analysis Cost

Finding 5. The strongest rankers are also the cheapest. Deriving the projection and counting dependents takes at most 0.06 s per scenario, and transitive reach at most 0.15 s. The learned engines’ forward pass takes milliseconds, but their node features take minutes on large architectures, dominated by the Connectivity Degradation Index.

Table 11: Per-stage latency of the inference pipeline across graph sizes (CPU, median of 3 runs; 5 for the forward pass). The analysis stage is stable across repeats (p10–p90 within 1% of the median), while the forward pass is dominated by interpreter and dispatch overhead. The 249-node forward time includes first-call warm-up and is an upper bound.

V	E	Analyze	Graph →	HGT	Analyze :	Forward
		(s)	Tensor (s)	Forward (ms)	Forward	p10–p90 (ms)
249	1,127	1.74	0.010	26.5	66×	13.0–34.4
499	2,402	8.32	0.022	16.4	509×	15.6–16.4
999	6,422	44.54	0.056	21.1	2,108×	19.1–36.5
1,998	19,301	239.34	0.157	56.2	4,259×	43.8–57.8

At 2,000 components the HGT forward pass takes 56 ms against 239 s for structural analysis (Table 11). Because the analysis stage produces the node features the forward pass consumes, end-to-end evaluation of an unseen 2,000-component architecture takes about four minutes, of which the learned model is 0.02%; the 56 ms is the marginal cost of re-scoring an already-analysed graph. The dominant term is the Connectivity Degradation Index, computed for every node in the main connected component.

Across the corpus, the complete analysis gate (structural analysis plus 18 anti-pattern detectors) runs in 0.16–79.3 s per scenario, 2.0–17.7× (median 5.6×) the five-seed cascade labeling sweep measured in the same session; the premium tracks the size of the derived projection (Supplementary Section S28). On raw CPU time, running the simulator directly is therefore cheaper than the full analysis gate. The dependency counts are cheaper than both: on the largest scenario, Enterprise, the projection takes 0.05 s, InDeg under 1 ms and Reach 0.09 s, against 4.5 s for the simulator and 79.3 s for the gate (`results/dependency_count_cost.json`). Training the four learned arms once took 7.7 CPU-hours.

8. Discussion, Threats to Validity, and Limitations

8.1. Implications for Practice and Research

For architects: make the dependencies explicit, then count. The most useful single step is to model the architecture with typed entities and derive its dependency projection. A CI gate can then rank every Application by the number of components that depend on it, in well under a second, with no training data, no runtime telemetry and no declared QoS profile, and it explains itself, for example “37 components depend on this service”. On held-out architectures this ranks simulated cascade impact at $\rho = 0.764$ and identifies inert components with 94% balanced accuracy; on the five system models transitive reach reaches $\rho = 0.938$. The explanation layer then names the remediation class for each flagged component: replication for a single point of failure, circuit breakers for a cascade hub, decoupling for a coupling bottleneck.

For researchers: judge learned engines against the simplest faithful baseline. Graph learning on architecture graphs is usually compared with centrality indices, and against those it looks strong here too: hybrids beat betweenness significantly, and learned engines transfer. Their accuracy, however, comes from per-component features: a gradient-boosted regressor with no graph model matches them, and neither relation typing, message passing nor the QoS edge channel adds anything measurable (Section 7.2). Against a count of dependents on the same projection it does not yet add accuracy on a reachability oracle. The engines receive in-degree among their inputs and still do not reproduce its ranking, which points to two concrete research directions: objectives and normalizations that preserve simple strong signals across graphs of different sizes, and targets on which structure alone is not enough. Behavioral oracles that express queueing, deadline misses or durability replay are the natural next targets; the matched control and the QoS-attribution controls of this study are the protocol for testing whether learning and QoS semantics add value there.

Choosing an instrument. Table 12 summarizes where each instrument fits on current evidence.

Table 12: How the instruments in the SaG portfolio are best used, given the evidence in Section 7.

Instrument	Context	Role and evidence
InDeg / Reach (dependency counts)	Default CI gate for any architecture	Training-free; $\rho = 0.764 / 0.732$ under LOSO, 0.863 / 0.938 on the system models; inert components identified at 94% balanced accuracy; under 0.15 s per scenario.
Hybrid-HGT / Hybrid-GAT	Learned ranking over structural cues	Significantly above the registered closed-form engine (+0.103 / +0.130, 11/12 folds); not above dependency counts.
Learned model on SaG features	Transfer across generators; richer oracles	Zero-shot GAT 0.831, GAT-QoS 0.805, HGT-QoS 0.760, feature-only regressor 0.757 (PR-AUC 0.71–0.81); an untyped network or gradient boosting suffices.
RM explanation layer	Remediation discussion	ISO/IEC 25010 attribution (Availability, Fault Tolerance, Maintainability); a design proposal awaiting validation.

Where learned engines help. Against the closed-form engine, the learned engines gain most on irregular meshes (Healthcare, IoT Smart City, ATM, Microservices) and on symmetric stars that create betweenness ties (EdgeX), and lose ground where the closed-form engine is strongest (Enterprise, Real-Time Gaming). The pattern belongs to the features, not to message passing:

- the per-node **GAT-QoS** and the message-passing **HGT-QoS** rise and fall on the same folds (Spearman 0.71 across folds);
- every learned model falls short on Enterprise (0.407–0.533 against 0.795);
- the share of the graph within HGT’s receptive field predicts neither its per-fold accuracy nor its gain over **Topo-QoS** ($|\rho| \leq 0.20$ across folds).

Each observation rests on one to three folds or systems; the replication repository tabulates them as working hypotheses.

Computational sustainability. Pre-deployment analysis avoids provisioning staging clusters for chaos sweeps [22, 24], which we state as infrastructure avoidance rather than a measured energy saving. Estimated at the platform’s base power (28 W) over wall-clock time, one pass of the full analysis gate over all twelve

scenarios costs about 0.83 Wh and training the four learned arms once about 0.22 kWh; the dependency counts cost a negligible fraction of either. Direct RAPL/NVML measurement [23, 98] would replace these estimates.

8.2. Threats to Validity

Construct validity. All labels are simulator-derived rather than observed failures. The primary oracle shares its dependency rules with the projection (Section 4.4), which is exactly why dependency counts rank it well; a ranking validated on this oracle measures how far a failure reaches through declared dependencies, not what it costs in production. The behavioral queue-flow oracle agrees with it at $\rho = 0.627$ (Section 4.3), and retargeting the evaluation on it is the next experiment. The five system models are the most judgement-laden input: each was written by one author and no second modeler has re-derived them. The replication package includes a re-modeling protocol and an agreement tool (`reproduce/model_agreement.py`).

Internal validity. Predictors consume G_{analysis} and oracles traverse $G_{\text{structural}}$, which CI enforces. Substrate, training set, depth and early stopping are matched across learned arms, and all typing conclusions rest on the capacity- and channel-matched control. The synthetic generator couples QoS to topology by design; the QoS-independent control corpus shows that this coupling does not produce the closed-form result (Section 7.1.2). The untyped arms receive no messages at the Applications they score (Section 6.2), so the matched control compares typed message passing with per-component learning, not two message-passing architectures. The attribution controls of Section 7.2 were added after the matched control’s result. None is registered: the feature-only regressor was declared post hoc in Amendment 3 and, like the others, first run under Amendment 8; they are reported as exploratory with their own Holm correction. The registered directionality control, HGT-QoS-U, removes HGT’s reverse pass and with it every message into an Application. It matches HGT-QoS with 103,725 fewer parameters ($p = 0.91$), so directionality does not confound the typing result, and HGT’s message passing contributes nothing measurable. The capacity-only control registered alongside it, GAT-w, gives the same answer ($p = 0.68$), so every model arm of Amendment 2 has been run; only its label-side sweep has not (Section 6.4). No hyperparameter was tuned on an evaluation split.

Robustness to free parameters. The oracle’s threshold and damping move label rankings by at most 0.14 in mean Spearman and the closed-form engine’s accuracy within 0.50–0.58 (Section 4.3). Under Morris screening only two of the explanation layer’s ten declared constants have appreciable influence ($\mu^* \approx 0.13$ against ≤ 0.025), and no setting of the topic-weight or QoS sub-weight constants changes any reported comparison (Supplementary Sections S1–S4).

External validity. The synthetic corpus comes from one generator family, and the system models are small (22–41 applications). Scaling the learned engines beyond 2,000 nodes would need incremental caching or mini-batching [99]; InDeg is linear in the size of the projection.

Conclusion validity and repeatability. LOSO folds share training scenarios, so p -values are nominal [94] and are read alongside fold-level sign counts and intervals. Training-free cells reproduce exactly across devices, and the Amendment 7 harness reproduced every published **Topo-QoS** fold value. Learned cells move across code revisions and devices, by up to 0.172 in a single fold mean and 0.041 in the HGT-QoS mean, so every comparison is made within one sweep and learned figures are reported against the released artifacts (`reproduce/rerun_drift.py`). InDeg’s margins over the learned and hybrid engines (+0.08 to +0.14 in mean ρ , won on 10–11 of 12 folds) exceed the mean-level drift; the typing effect and the attribution effects of RQ2 (−0.014; +0.095 for the QoS node columns and −0.023 for the edge channel) do not, which is one more reason to read them as descriptive. One source of run-to-run variation is now identified: the Connectivity Degradation Index breaks degree ties in set-iteration order, which Python salts per process (Supplementary Section S31).

8.3. Limitations and Future Work

The explanation layer’s attributions have not been evaluated with developers or against the outcome of the repairs they recommend, and no published learned-criticality model (FINDER [71], DrBC [72]) has been reproduced on this corpus. The agenda that follows from this study is: (1) retarget RQ1–RQ3 on the behavioral oracle, where the value of learning and of QoS semantics can be tested beyond reachability; (2) an

independent re-model of at least two of the five systems, with inter-modeler agreement reported; (3) learned objectives that preserve dependency counts, and a substrate or architecture on which messages reach every scored component, such as reverse edges for the untyped engines or the Application–Library projection, so that message passing is tested as a mechanism rather than left to edge direction; (4) synchronous call edges and a backward-propagating oracle, so that RPC architectures can be modeled natively; (5) extracting system models from deployment manifests; (6) validating rankings and recommended repairs against production incident data; and (7) registering future analysis plans with a third-party timestamp.

9. Conclusion

Cascading-failure risk in publish–subscribe systems hides in dependencies that no single component declares. SaG makes those dependencies explicit, and once they are explicit, risk becomes measurable before deployment. On twelve held-out synthetic architectures, counting a component’s dependents on SaG’s projection ranks its simulated cascade impact at $\rho = 0.764$, above the registered betweenness engine on every fold and above every learned and hybrid engine, and it separates components whose failure reaches no one with 94% balanced accuracy. On five independently authored models of open-source systems, dependency counts reach $\rho = 0.86$ – 0.94 with no training, and learned models trained only on synthetic data transfer at 0.757 – 0.831 . Hybrid engines are the only learned engines that significantly beat the closed-form engine. Matched controls trace learned accuracy to the per-component features SaG extracts: relation-specific parameters, message passing and the QoS edge encoding add nothing measurable, and a gradient-boosted regressor on the same features matches the neural engines. Registered controls attribute the closed-form gain to the dependency projection rather than to declared QoS contracts.

For practice, the lesson is concrete: model the architecture’s entities and derive its dependencies, and a CI gate can rank systemic risk in well under a second, explain each ranking by the components that depend on it, and name a remediation class for each flagged component. Teams do not need training data, runtime telemetry or complete QoS declarations to start.

For research, the study offers a benchmark, a torch-free harness and a registered protocol that make such claims checkable, and it sets a clear bar: learned architecture analyzers should be judged against the simplest faithful baseline on the same graph. The next steps are behavioral oracles on which learning and QoS semantics can show what they add, a substrate on which messages reach every scored component, independent re-models of the system models, and validation against production incidents. Because the corpus regenerates byte-identically and every reported value is reconciled against released artifacts, others can extend or challenge these results directly.

Declarations

CRediT Authorship Contribution Statement. **Ibrahim Onuralp Yigit:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Feza Buzluca:** Conceptualization, Methodology, Validation, Writing – review and editing, Supervision. **Declaration of Competing Interest.** The authors declare no competing financial interests or personal relationships that could have influenced this work. **Funding.** This research received no external grant. **Data Availability.** The replication package (datasets, harnesses, checkpoints, scripts) is available on Zenodo under DOI 10.5281/zenodo.14922108 [100] with `uv/pip` environments. The public repository documents every experiment reported here — protocol, hyperparameters, reproduction command, artifacts and extended results — at <https://github.com/onuralpyigit/software-as-a-graph/tree/jss-submission-v4/docs/research/jss/experiments>. Synthetic datasets regenerate byte-identically. The deposit ships all artifacts backing reported tables as a dated bundle (`SaG_JSS_Results_<stamp>`) with a `MANIFEST.json` recording SHA-256 digests, commit hashes, and corpus provenance. Four supplementary artifacts predate provenance stamping and carry no commit or corpus digest: `atm_scale_sweep_v3.json` (Supplementary Section S6), `qos_label_ablation.json` (Section 4.3), `threshold_sensitivity_v3.json` (Supplementary Section S3) and `topic_weight_sensitivity_v3.json` (Supplementary Section S1); their correspondence to the corpus is

asserted by the bundle rather than recorded in the file. The Amendment 7 artifacts (`tf_baselines.json`, `qos_attribution_controls.json`, `qos_indep_corpus.json`, `topo_substrate_check.json`, `oracle_param_sensitivity.json`, `system_model_descriptives.json`, `dependency_count_cost.json`) are provenance-stamped and regenerate without GPU or database from `reproduce/training_free_suite.py`. The verification script (`reproduce/reconcile_manuscript.py`) runs standalone against the deposit, mechanically verifying every reported table figure in the manuscript and supplement against the JSON artifact that produced it.

Declaration of Generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Anthropic’s Claude to assist with typesetting and LaTeX formatting, with drafting and editing of the manuscript text during revision, and with the development of analysis and reporting scripts in the replication package. After using this tool the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. The study design, the choice of experiments, the interpretation of results, and all scientific claims are the authors’ own.

References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Science Robotics* 7 (66) (2022) eabm6074.
- [2] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB)*, 2011.
- [3] Object Management Group, Data distribution service (dds), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).
- [4] OASIS, MQTT version 5.0, OASIS Standard, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (accessed 9 September 2026) (2019).
- [5] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, today, and tomorrow, in: *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, O’Reilly Media, 2015.
- [7] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, *ACM Computing Surveys* 35 (2) (2003) 114–131.
- [8] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, *Physical Review E* 66 (2002) 065102(R).
- [9] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, *Nature* 464 (2010) 1025–1028.
- [10] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, *Nature* 406 (2000) 378–382.
- [11] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, *IEEE Transactions on Dependable and Secure Computing* 1 (1) (2004) 11–33.
- [12] L. Bass, P. Clements, R. Kazman, *Software Architecture in Practice*, 3rd Edition, Addison-Wesley, 2012.
- [13] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, J. Carriere, The architecture tradeoff analysis method, in: *Proc. 4th IEEE Int. Conf. on Engineering of Complex Computer Systems (ICECCS)*, 1998, pp. 68–78.
- [14] SonarSource, Clean as you code, SonarQube documentation, <https://docs.sonarsource.com/sonarqube-server/latest/core-concepts/clean-as-you-code/introduction/> (accessed 9 September 2026) (2024).
- [15] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, *IEEE Transactions on Software Engineering* 20 (6) (1994) 476–493.
- [16] D. E. Perry, A. L. Wolf, Foundations for the study of software architecture, *ACM SIGSOFT Software Engineering Notes* 17 (4) (1992) 40–52.
- [17] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: *Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR)*, 2009, pp. 255–258.
- [18] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, *IEEE Software* 33 (3) (2016) 35–41.
- [19] L. C. Freeman, A set of measures of centrality based on betweenness, *Sociometry* 40 (1) (1977) 35–41.
- [20] U. Brandes, A faster algorithm for betweenness centrality, *Journal of Mathematical Sociology* 25 (2) (2001) 163–177.
- [21] I. O. Yigit, F. Buzluca, A graph-based dependency analysis method for identifying critical components in distributed publish–subscribe systems, in: *Proc. IEEE Int. Conf. on Recent Advances in Systems Science and Engineering (RASSE)*, 2025, pp. 1–8. doi:10.1109/RASSE64831.2025.11315354.
- [22] C. Calero, M. Piattini (Eds.), *Green in Software Engineering*, Springer, Cham, Switzerland, 2015. doi:10.1007/978-3-319-08581-4.
- [23] L. Lanelongue, J. Grealey, M. Inouye, Green algorithms: Quantifying the carbon footprint of computation, *Advanced Science* 8 (12) (2021) 2100707. doi:10.1002/advs.202100707.
- [24] R. Verdecchia, J. Sallou, L. Cruz, A systematic review of Green AI, *WIREs Data Mining and Knowledge Discovery* 13 (4) (2023) e1507. doi:10.1002/widm.1507.
- [25] R. C. Cheung, A user-oriented software reliability model, *IEEE Transactions on Software Engineering* SE-6 (2) (1980) 118–125.
- [26] K. Goseva-Popstojanova, K. S. Trivedi, Architecture-based approach to reliability assessment of software systems, *Performance Evaluation* 45 (2–3) (2001) 179–204.

- [27] A. Immonen, E. Niemelä, Survey of reliability and availability prediction methods from the architectural perspective, *Software and Systems Modeling* 7 (1) (2008) 49–65.
- [28] S. Becker, H. Koziulek, R. Reussner, The Palladio component model for model-driven performance prediction, *Journal of Systems and Software* 82 (1) (2009) 3–22.
- [29] F. Brosch, H. Koziulek, B. Buhnova, R. Reussner, Architecture-based reliability prediction with the Palladio component model, *IEEE Transactions on Software Engineering* 38 (6) (2012) 1319–1339. doi:10.1109/TSE.2011.94.
- [30] G. Franks, T. Al-Omari, M. Woodside, O. Das, S. Derisavi, Enhanced modeling and solution of layered queueing networks, *IEEE Transactions on Software Engineering* 35 (2) (2009) 148–161.
- [31] J. Delange, P. H. Feiler, Architecture fault modeling with the AADL error-model annex, in: 2014 40th EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA), IEEE, 2014, pp. 361–368. doi:10.1109/SEAA.2014.20.
- [32] Y. Papadopoulos, J. A. McDermid, Hierarchically performed hazard origin and propagation studies, in: *Computer Safety, Reliability and Security (SAFECOMP)*, Vol. 1698 of Lecture Notes in Computer Science, Springer, 1999, pp. 139–152. doi:10.1007/3-540-48249-0_13.
- [33] C. S. Meiklejohn, A. Estrada, Y. Song, H. Miller, R. Padhye, Service-level fault injection testing, in: *Proc. ACM Symposium on Cloud Computing (SoCC)*, 2021. doi:10.1145/3472883.3487005.
- [34] P. Alvaro, J. Rosen, J. M. Hellerstein, Lineage-driven fault injection, in: *Proc. ACM SIGMOD International Conference on Management of Data*, 2015, pp. 331–346. doi:10.1145/2723372.2723711.
- [35] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, C. Delimitrou, Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices, in: *Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.
- [36] Y. Gan, M. Liang, S. Dev, D. Lo, C. Delimitrou, Sage: Practical and scalable ML-driven performance debugging in microservices, in: *Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.
- [37] L. Wu, J. Tordsson, E. Elmroth, O. Kao, MicroRCA: Root cause localization of performance issues in microservices, in: *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2020.
- [38] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, D. Pei, Practical root cause localization for microservice systems via trace analysis, in: *Proc. IEEE/ACM Int. Symposium on Quality of Service (IWQoS)*, 2021.
- [39] Y. Meng, S. Zhang, Y. Sun, R. Zhang, Z. Hu, Y. Zhang, C. Jia, Z. Wang, D. Pei, Localizing failure root causes in a microservice through causality inference, in: *Proc. IEEE/ACM 28th International Symposium on Quality of Service (IWQoS)*, 2020. doi:10.1109/IWQoS49365.2020.9213058.
- [40] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, D. Zhang, DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning, in: *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2022.
- [41] C. Lee, T. Yang, Z. Chen, Y. Su, Y. Yang, M. R. Lyu, Eadro: An end-to-end troubleshooting framework for microservices on multi-source data, in: *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2023.
- [42] J. Soldani, A. Brogi, Anomaly detection and failure root cause analysis in (micro)service-based cloud applications: A survey, *ACM Computing Surveys* 55 (3) (2022). doi:10.1145/3501297.
- [43] S. Zhang, S. Xia, W. Fan, B. Shi, X. Xiong, Z. Zhong, M. Ma, Y. Sun, D. Pei, Failure diagnosis in microservice systems: A comprehensive survey and analysis, *ACM Transactions on Software Engineering and Methodology* (2025). doi:10.1145/3715005.
- [44] S. Luo, H. Xu, C. Lu, K. Ye, G. Xu, L. Zhang, Y. Ding, J. He, C. Xu, Characterizing microservice dependency and performance: Alibaba trace analysis, in: *Proc. ACM Symposium on Cloud Computing (SoCC)*, 2021. doi:10.1145/3472883.3487003.
- [45] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, *IEEE Transactions on Software Engineering* 47 (2) (2021) 243–260.
- [46] S. A. Bohner, R. S. Arnold, *Software Change Impact Analysis*, IEEE Computer Society Press, Los Alamitos, CA, 1996.
- [47] S. Esparrachiarri, T. Reilly, A. Rentz, Tracking and controlling microservice dependencies, *ACM Queue* 16 (4) (2018). doi:10.1145/3277539.3277541.
- [48] X. Yang, K. Tang, X. Yao, A learning-to-rank approach to software defect prediction, *IEEE Transactions on Reliability* 64 (1) (2015) 234–246.
- [49] T. J. McCabe, A complexity measure, *IEEE Transactions on Software Engineering SE-2* (4) (1976) 308–320.
- [50] N. Fenton, J. Bieman, *Software Metrics: A Rigorous and Practical Approach*, 3rd Edition, CRC Press, 2014.
- [51] V. R. Basili, L. C. Briand, W. L. Melo, A validation of object-oriented design metrics as quality indicators, *IEEE Transactions on Software Engineering* 22 (10) (1996) 751–761.
- [52] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: *Proc. 27th Int. Conf. on Software Engineering (ICSE)*, 2005, pp. 580–586.
- [53] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: *Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE)*, 2007.
- [54] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, *IEEE Transactions on Software Engineering* 33 (1) (2007) 2–13.
- [55] V. Bushong, D. Das, A. Al Maruf, T. Cerny, Using static analysis to address microservice architecture reconstruction, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2021. doi:10.1109/ASE51524.2021.9678749.
- [56] S. Schneider, A. Bakhtin, X. Li, J. Soldani, A. Brogi, T. Cerny, R. Scandariato, D. Taibi, Comparison of static analysis architecture recovery tools for microservice applications, *Empirical Software Engineering* 30 (5), preprint: arXiv:2412.08352 (2025).
- [57] A. Santos, A. Cunha, N. Macedo, Static-time extraction and analysis of the ROS computation graph, in: *Proc. Third IEEE International Conference on Robotic Computing (IRC)*, 2019, pp. 62–69. doi:10.1109/IRC.2019.00018.
- [58] C. S. Timperley, T. Dürschmid, B. Schmerl, D. Garlan, C. Le Goues, ROSDiscover: Statically detecting run-time architecture misconfigurations in robotics systems, in: *Proc. 19th IEEE International Conference on Software Architecture (ICSA)*, 2022, pp. 112–123. doi:10.1109/ICSA53651.2022.00019.
- [59] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: *Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA)*, LNCS 5581, 2009, pp. 146–162.
- [60] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, *IEEE Software* 35 (3) (2018) 56–62.

- [61] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, *Journal of Systems and Software* 101 (2015) 193–220.
- [62] J. Humble, D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
- [63] L. Chen, Continuous delivery: Huge benefits, but challenges too, *IEEE Software* 32 (2) (2015) 50–54.
- [64] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, Tech. rep., International Organization for Standardization (2023).
- [65] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).
- [66] International Organization for Standardization, ISO/IEC 25023:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of system and software product quality, Tech. rep., International Organization for Standardization (2016).
- [67] International Organization for Standardization, ISO/IEC 25021:2012 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality measure elements, Tech. rep., International Organization for Standardization (2012).
- [68] T. L. Saaty, *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*, McGraw-Hill, 1980.
- [69] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, *Computer Networks and ISDN Systems* 30 (1–7) (1998) 107–117.
- [70] M. E. J. Newman, *Networks: An Introduction*, Oxford University Press, 2010.
- [71] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, *Nature Machine Intelligence* 2 (2020) 317–324.
- [72] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: *Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2019, pp. 559–568.
- [73] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, *Datasets and Benchmarks Track*, 2024, arXiv:2402.02827.
- [74] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.
- [75] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 1024–1034.
- [76] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2018.
- [77] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: *Proc. European Semantic Web Conference (ESWC)*, 2018, pp. 593–607.
- [78] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: *Proc. The Web Conference (WWW)*, 2019, pp. 2022–2032.
- [79] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: *Proc. The Web Conference (WWW)*, 2020, pp. 2704–2710.
- [80] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: *Proc. The Web Conference (WWW)*, 2020, pp. 2331–2341.
- [81] G. Khodabandeh, A. Ezaz, M. Babaei, N. Ezzati-Jivan, Utilizing graph neural networks for effective link prediction in microservice architectures, in: *Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE)*, 2025.
- [82] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, GNNExplainer: Generating explanations for graph neural networks, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32, 2019, pp. 9244–9255.
- [83] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, X. Zhang, Parameterized explainer for graph neural network, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, 2020, pp. 19620–19631.
- [84] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*, Morgan Kaufmann, 1988.
- [85] G. Beliakov, A. Pradera, T. Calvo, *Aggregation functions: A guide for practitioners*, *Studies in Fuzziness and Soft Computing* 221 (2007).
- [86] R. R. Yager, On ordered weighted averaging aggregation operators in multicriteria decisionmaking, *IEEE Transactions on Systems, Man, and Cybernetics* 18 (1) (1988) 183–190.
- [87] G. H. Hardy, J. E. Littlewood, G. Pólya, *Inequalities*, 2nd Edition, Cambridge University Press, 1952.
- [88] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch geometric, in: *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [89] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: *Proc. 25th Int. Conf. on Machine Learning (ICML)*, 2008, pp. 1192–1199.
- [90] Team SimPy, *Simpy: Discrete event simulation for Python*, Software, <https://simpy.readthedocs.io> (accessed 9 September 2026) (2020).
- [91] F. Wilcoxon, Individual comparisons by ranking methods, *Biometrics Bulletin* 1 (6) (1945) 80–83.
- [92] J. Demšar, Statistical comparisons of classifiers over multiple data sets, *Journal of Machine Learning Research* 7 (2006) 1–30.
- [93] B. Efron, R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall, 1993.
- [94] C. Nadeau, Y. Bengio, Inference for the generalization error, *Machine Learning* 52 (2003) 239–281. doi:10.1023/A:1024068626366.
- [95] A. Benavoli, G. Corani, J. Demšar, M. Zaffalon, Time for a change: A tutorial for comparing multiple classifiers through Bayesian analysis, *Journal of Machine Learning Research* 18 (77) (2017) 1–36.
- [96] D. Lakens, Equivalence tests: A practical primer for *t* tests, correlations, and meta-analyses, *Social Psychological and Personality Science* 8 (4) (2017) 355–362. doi:10.1177/1948550617697177.
- [97] A. Gelman, H. Stern, The difference between “significant” and “not significant” is not itself statistically significant, *The American Statistician* 60 (4) (2006) 328–331. doi:10.1198/000313006X152649.
- [98] V. Schmidt, K. Goyal, A. Joshi, B. Feld, L. Conell, N. Laskaris, D. Sarthou, H. Verreault, J. Blank, S. Zhang, Codecarbon: Estimate and track carbon emissions from machine learning computing, *Journal of Open Source Software* (2021).

- 999 [99] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, V. Prasanna, GraphSAINT: Graph sampling based inductive engine, in:
1000 Proc. International Conference on Learning Representations (ICLR), 2020.
- 1001 [100] I. O. Yigit, F. Buzluca, [dataset] software-as-a-graph: Replication package (datasets, generator configurations, simulation
1002 harnesses, model checkpoints, and analysis scripts), <https://doi.org/10.5281/zenodo.14922108> (2026). doi:10.5281/
1003 zenodo.14922108.